Neural Networks 206 (2027) 109831

Contents lists available at ScienceDirect

Neural Networks

journal homepage: www.elsevier.com/locate/neunet

Full Length Article

STMH-Xception: Sequential multi-head residual learning for robust underwater acoustic target recognition

Chenhong Yan [a,1], Shefeng Yan [b], Ruobin Gao [a,*], Yang Yu [a,c,1], Guang Pan [a], Jinpeng Xu [a], Ponnuthurai N. Suganthan [d]

[a] School of Marine Science and Technology, Northwestern Polytechnical University, Xi'an, 710072, China
[b] Institute of AI for Industries, Chinese Academy of Sciences, Nanjing, 211135, China
[c] Research & Development Institute of Northwestern Polytechnical University in Shenzhen, Nanshan District, Shenzhen, 518063, China
[d] Department of Computer Science and Engineering, College of Engineering, Qatar University, Doha, 2713, Qatar

ARTICLE INFO

Keywords:
Underwater acoustic target recognition
Sequential multi-head learning
Xception
Residual learning
Gradient boosting

ABSTRACT

Underwater Acoustic Target Recognition (UATR) remains a formidable challenge due to highly non-stationary marine environments and the complex, overlapping acoustic signatures of different vessels. Deep Neural Networks (DNNs) typically employ a single-head optimization paradigm, rendering them susceptible to overfitting individual-vessel noise rather than learning robust, category-consistent features. To bridge this gap, we propose a novel framework, the Sequentially Trained Multi-Head Xception (STMH-Xception). Drawing inspiration from gradient boosting, we introduce a step-wise STMH strategy that integrates deeply supervised auxiliary heads into a depthwise separable convolutional backbone. Unlike standard joint-training multi-task learning, STMH employs a sequential freezing protocol whereby each subsequent head targets and corrects the residual representation errors of its predecessors. This "coarse-to-fine" learning scheme drives the network to extract a highly discriminative feature hierarchy while mitigating inter-head gradient conflicts associated with joint multi-head optimization. Validated on the ShipsEar and DeepShip datasets, the proposed STMH-Xception increases the classification accuracy from 68.72% to 73.10% and from 72.89% to 76.14%, respectively, compared to the baseline. Supported by statistical significance tests and generalizability analyses across diverse CNN encoders, the STMH strategy emerges as a highly efficient, CNN-backbone-agnostic optimization paradigm. The additional parameter and computational cost introduced by the auxiliary heads remain small, making the framework promising for resource-constrained UATR scenarios.

1. Introduction

Underwater acoustic target recognition (UATR) (Jin et al., 2023) aims to identify vessel categories from ship-radiated noise and has long been regarded as a critical enabling capability for maritime situational awareness, safety-related operations, and ocean resource monitoring (Dong et al., 2021). In contrast to controlled terrestrial audio processing, recognizing targets in the underwater environment presents unique and formidable challenges. The underwater acoustic channel is characterized by severe multipath effects, frequency-dependent attenuation, and complex, non-stationary marine background noise (Gao et al., 2025b; Wang et al., 2024). These factors result in acoustic signals that are highly time-varying and distorted. Early recognition systems predominately relied on handcrafted acoustic features combined with conventional machine learning algorithms (Hummel et al., 2024). Despite incorporating extensive expert domain knowledge, these approaches often demonstrated suboptimal generalization when applied to complex real-world continuous data.

With the increasing availability of sensing platforms and public datasets (Irfan et al., 2021; Santos-Domínguez et al., 2016), Deep Learning (DL) has shifted the paradigm from feature engineering to data-driven representation learning, gradually becoming the mainstream solution for UATR (Jin et al., 2025, 2020). Convolutional Neural Networks (CNNs) (Li & Yang, 2021) are the most widely adopted architectures, as their inductive biases align perfectly with the local time-frequency patterns of underwater signals. Consequently, many pipelines first convert ship-radiated noise into 2-D time-frequency representations—such as Mel-spectrograms (Liu et al., 2021), Constant Q Transform (CQT) (Xie et al., 2023), Low-Frequency Analyzer and Recorder (LOFAR) spectrum (Chen et al., 2021), and Mel-Frequency Cepstrum Coefficients (MFCC)

* Corresponding author.
E-mail address: gaoruobin@nwpu.edu.cn (R. Gao).
[1] This is the first author footnote.

https://doi.org/10.1016/j.neunet.2026.109831
Received 12 April 2026; Received in revised form 27 July 2026; Accepted 29 October 2026
Available online 2 November 2026
0893-6080/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

(Zhang et al., 2016)—and then employ CNNs for end-to-end recognition (Yan et al., 2023).

Despite their undeniable effectiveness, most CNN-based UATR pipelines are optimized under a single-head, single-objective formulation, where one classifier is trained to fit all samples simultaneously. A fundamental challenge in UATR is that a vessel's acoustic signature is influenced not only by its category label but also by individual factors such as hull structure, machinery, and propulsion configuration. Even vessels of the same category may exhibit significant variation in these attributes, while vessels from different categories may share similar propulsion systems, leading to overlapping acoustic patterns (Fang et al., 2025). Under a single-head formulation, deep models often take "shortcuts" by overfitting to these individual-vessel-specific features rather than learning consistent, category-level discriminative patterns. This leads to increased intra-class variability and reduced inter-class separability, causing severe performance drops when the model encounters unseen vessels or shifts in data distribution.

To mitigate the variance caused by these vessel-specific factors and improve robustness, ensemble learning has been widely investigated as a powerful solution (Deng et al., 2023; Najamuddin et al., 2026). However, conventional ensemble methods typically require training and storing multiple independent models, which imposes prohibitive computational and memory costs (Ye et al., 2024). This is particularly undesirable for resource-constrained, real-time marine systems, such as Autonomous Underwater Vehicles (AUVs). Alternatively, Deep Supervision and standard Multi-Task Learning (MTL) attempt to incorporate multiple branches within a single network. Yet, standard MTL employs a parallel training paradigm where all auxiliary branches are updated simultaneously. When the available training data is limited and potentially biased, this parallel optimization frequently induces gradient conflicts, where shallow and deep heads pull the shared backbone in diverging directions.

The complexity of UATR is further exacerbated by substantial variations in real-world data availability. In restricted or specialized marine operations, acoustic data collection is often highly limited. This creates a "small-data" regime characterized by scarce samples and class imbalance (Santos-Domínguez et al., 2016), which easily induces the aforementioned overfitting to vessel-specific acoustic characteristics. Conversely, long-term continuous marine monitoring generates large-scale acoustic recordings (Irfan et al., 2021). Such data introduce pronounced intra-class variations and persistent, non-stationary background noise, frequently leading to performance saturation in standard networks. The ShipsEar and DeepShip datasets adopted in this study represent the data-scarce and large-scale regimes described above, respectively. Nevertheless, both benchmarks are designed for single-target ship-radiated noise classification. Accordingly, the present work focuses on single-target UATR, while multi-target overlapping scenarios are beyond the scope of the current evaluation. Within this scope, a robust UATR framework should adapt to both extremes, demanding a mechanism that retains the robustness benefits of ensemble-like error correction, prevents gradient conflicts, and remains highly efficient for real-time deployment.

Addressing these critical requirements, and building upon our previous ResNet18-based exploration of sequential multi-head mechanisms (Yan et al., 2026), this paper proposes the Sequentially Trained Multi-Head Xception (STMH-Xception) framework. At the representation level, we refine the Short-Time Fourier Transform (STFT) parameters to extract higher-resolution Mel-spectrograms tailored to capture fine-grained, non-stationary spectral characteristics of underwater targets. Architecturally, we employ a depthwise separable Xception backbone, whose separable convolutions decouple spatial and cross-channel feature extraction. Six lightweight heads are aligned with the Stem, Entry, Middle, and Exit stages to capture hierarchical acoustic representations while introducing minimal additional overhead. During sequential training, we introduce a dynamic recomputation mechanism so that the reference logits from previously trained heads are recomputed using the evolving backbone features. This design maintains consistent sequential refinement as the shared backbone continues to adapt. Consequently, the proposed STMH-Xception improves recognition performance while exhibiting stable optimization behavior and low parameter and floating-point operations (FLOPs) overhead in our experiments. The main contributions of this article are summarized as follows:

1. We formulate UATR from a boosting-inspired error-correction perspective and propose the STMH-Xception framework, in which six lightweight auxiliary heads are placed according to the major structural transitions across the heterogeneous Stem, Entry, Middle, and Exit flows of Xception. By embedding this architecture-aware sequential multi-head design within a single network, the framework fosters complementary decision-making and balances representational capacity with computational efficiency.
2. We introduce the STMH training strategy. By modeling deep supervision as a sequential residual refinement process, it mitigates the gradient conflict issue commonly observed in standard joint multi-task learning and reduces the network's tendency to overfit individual-vessel-specific noise.
3. We provide comprehensive cross-scale validations. The proposed method achieves accuracies of 73.10% on the data-scarce ShipsEar benchmark and 76.14% on the large-scale DeepShip dataset, consistently outperforming evaluated high-capacity architectures including DenseNets and Transformers.
4. We present statistical significance tests and generalizability analyses, confirming that the STMH protocol yields consistent performance improvements across diverse CNN backbones.

The remainder of this article is organized as follows. Section 2 reviews related work on UATR and efficient ensemble strategies. Section 3 introduces the proposed STMH-Xception framework and the sequential training methodology. Section 4 outlines the experimental setup, including dataset partitioning and implementation details. Section 5 presents the comprehensive comparative results, visual analyses, and ablation studies. Finally, Section 6 concludes the paper.

2. Related work

2.1. Underwater acoustic target recognition

Significant advancements in UATR have been achieved over the past decades. Early approaches predominantly relied on handcrafted acoustic features combined with conventional machine learning classifiers. For example, Das et al. (2013) utilized spectral cepstral features with a Gaussian mixture model, while Wang and Zeng (2014) employed wavelet analysis and the Hilbert-Huang transform coupled with a Support Vector Machine (SVM). However, manual feature extraction is heavily dependent on expert domain knowledge and inherently struggles to capture the immense variability of large-scale datasets, often resulting in suboptimal generalization in complex marine environments (Irfan et al., 2021).

With the proliferation of deep learning, CNNs have provided a highly effective data-driven alternative. Cao et al. (2018) combined CNNs with second-order pooling to capture complex time-frequency correlations, while Liu et al. (2021) employed 3-D Mel-spectrograms alongside a Convolutional Recurrent Neural Network (CRNN). To circumvent the limitations of rigid manual feature engineering, recent studies have introduced learnable front-end modules for adaptive feature extraction (Ren et al., 2022; Xie et al., 2022). Furthermore, to meet the strict efficiency constraints of real-world marine deployments, lightweight architectures utilizing asymmetric convolutions have been successfully developed to reduce computational overhead while maintaining high accuracy (Yan et al., 2023). Building upon this, techniques integrating multi-scale attention mechanisms (Yan et al., 2024; Zhou et al., 2023b) and auxiliary spatial information, such as source distance (Yang et al., 2024), have been explored to further enrich feature representations without introducing prohibitive computational burdens.

Acoustic-wave-based sensing has also enabled dynamic monitoring in related photoacoustic imaging applications (Lv et al., 2024). To further enhance generalization under severe domain shifts—caused by time-varying ocean conditions and non-stationary background noise—researchers have investigated robust optimization strategies. Xu et al. (2023a) introduced local masking and smoothness-based regularizers, while Zhou et al. (2023a) integrated denoising modules directly into the recognition pipeline. Furthermore, state-of-the-art generative approaches, such as diffusion models, have recently been proposed to explicitly model and reduce complex marine ambient noise distributions prior to classification (Zhang et al., 2025). Recently, Transformer-based models (Feng & Zhu, 2022) and Self-Supervised Learning (SSL) paradigms (Wang et al., 2022; Xu et al., 2023b) have gained traction for their ability to model long-range dependencies and leverage unlabeled data.

Despite these advancements, intra-class variability and inter-class commonality remain formidable challenges in UATR. Vessels within the same class often exhibit vastly different acoustic signatures, whereas vessels from different classes may share similar propulsion patterns. While advanced methods like adversarial domain adaptation (Zeng et al., 2020) and Mixture-of-Experts (MoE) (Xie et al., 2024) have shown promise in handling heterogeneous data distributions, they frequently introduce substantial computational overhead and heavy annotation burdens. This underscores the urgent need for lightweight, deployable frameworks that can mitigate individual biases and enhance generalization without deviating from standard supervised learning pipelines.

2.2. Efficient ensemble strategies

Ensemble learning is a well-established paradigm for improving generalization and uncertainty estimation by aggregating predictions from multiple diverse hypotheses (Dietterich, 2000). In deep learning, Deep Ensembles (Lakshminarayanan et al., 2017) demonstrate state-of-the-art robustness by training multiple randomly initialized networks independently. However, the computation and memory costs of deploying multiple heavy backbones are often prohibitive for resource-constrained applications like real-time AUV acoustic monitoring. To mitigate this, efficient alternatives such as Snapshot Ensembles (Huang et al., 2017) and Fast Geometric Ensembling (Garipov et al., 2018) create diverse models from a single training trajectory. While efficient, these methods primarily rely on stochasticity for diversity rather than an explicit mechanism to correct specific errors, limiting their ability to resolve the hard, non-stationary acoustic patterns inherent to UATR tasks.

A more targeted alternative is found in Boosting algorithms (e.g., AdaBoost, Gradient Boosting), which construct a strong classifier sequentially by focusing each new learner on the samples misclassified by previous ones (Freund & Schapire, 1997). Integrating this sequential error-correction principle into deep neural networks has been an active research area. Early works like Boosted CNNs (Moghimi et al., 2016) incorporated boosting weights into the loss function, while approaches like AdaNet (Cortes et al., 2017) structurally grow the network to minimize residual errors. Despite their theoretical appeal, most boosting-based deep models face practical bottlenecks. They either require training multiple separate networks sequentially (increasing latency) or rely on complex instance-reweighting schemes that are highly unstable with Stochastic Gradient Descent (SGD). To reconcile the sequential benefits of boosting with inference efficiency, multi-head architectures offer a structural middle ground. However, standard MTL (Ruder, 2017) optimizes heads simultaneously, lacking an explicit sequential dependency for iterative refinement. Cascading Networks (Cai & Vasconcelos, 2018) stack classifiers in stages, but rely on spatial resampling and repeated feature extraction, introducing significant procedural redundancy.

Recently, the concept of utilizing multiple output layers to progressively enhance representation capability has proven highly effective in randomized neural networks. For instance, multi-output architectures optimized via closed-form solutions have been successfully deployed in ensemble deep RVFL networks for tabular and electroencephalogram (EEG) signal classification (Gao et al., 2026; Malik et al., 2023; Shi et al., 2022). Furthermore, advanced stacked and dynamic ensemble variants of deep randomized networks have demonstrated exceptional robustness in complex medium-scale time-series and oceanographic wave height forecasting (Gao et al., 2025a,c). Similar hybrid multi-output approaches have been applied to standard Multilayer Perceptrons (MLPs) (Hamdy et al., 2026). However, adapting these sequential, boosting-like multi-output paradigms into fully backpropagation-trained CNNs for complex, non-stationary acoustic data remains largely unexplored.

Building upon our previous sequential multi-head study (Yan et al., 2026), the present work develops STMH-Xception as an architecture-aware instantiation of boosting-inspired residual refinement for a depthwise separable CNN. The extension is not a simple replacement of ResNet18 with Xception. In the previous ResNet18 implementation, the regular residual-stage hierarchy provided natural attachment points for sequential heads. Xception instead contains heterogeneous Stem, Entry, Middle, and Exit flows whose representation changes are not uniformly distributed across network depth. STMH-Xception therefore maps the ordered residual-correction stages to these major structural transitions, placing heads more densely around the resolution- and channel-changing Entry Flow and more sparsely within the repetitive fixed-scale Middle Flow. This architecture-aware placement couples each cumulative-logit correction stage with a representationally distinct level of the Xception hierarchy. Residual refinement is performed through lightweight heads within one shared backbone, avoiding independent CNN learners, duplicated backbones, and explicit sample reweighting. To clarify the methodological position of STMH-Xception, Table 1 compares it with deep supervision, multi-exit networks, boosted CNNs, AdaNet, and our previous multi-head study.

3. Methodology

This section details the proposed STMH framework adapted for the Xception architecture. We begin by defining the overall workflow and the specific adaptations made to the Xception backbone. Subsequently, we elaborate on the multi-head integration strategy and the boosting-inspired sequential training protocol designed to progressively refine underwater acoustic target recognition.

At its core, the proposed STMH framework operates on the fundamental principle of “shared-backbone and frozen-head residual refinement”. Diverging from standard multi-task learning where simultaneous updates inherently induce severe optimization interference, our paradigm structurally isolates the training of each auxiliary classifier. By freezing the parameters of previously optimized heads, STMH preserves the learned classifiers while dynamically recomputing their outputs using the evolving backbone representations. At each stage, the model optimizes a single cumulative classification objective rather than multiple independently defined auxiliary losses, thereby avoiding the simultaneous competition among separate head-specific objectives in conventional joint multi-head learning.

3.1. Overall architecture

As illustrated in Fig. 1, the proposed STMH framework integrates signal preprocessing with a boosting-style deep learning architecture for UATR. The workflow is structurally organized into four key modules: signals acquisition, Mel-spectrogram extraction, a shared CNN backbone, and a sequential multi-head learning module. Critically, the STMH learning principle is designed to be CNN-backbone-agnostic, allowing it to be integrated with diverse CNN architectures that provide ordered intermediate feature representations. However, the specific attachment positions of the auxiliary heads are architecture-dependent and should be selected according to the structural transitions of the chosen backbone. Accordingly, in STMH-Xception, the auxiliary heads are aligned

Table 1
Conceptual comparison between STMH-Xception and related learning paradigms.

Paradigm	Typical mechanism	Essential distinction of STMH-Xception
Deep Mutual Learning (DML) (Zhang et al., 2018b)	Simultaneous optimization of peer networks using supervised and mutual distillation losses	STMH-Xception uses no bidirectional peer-distillation objective; its heads are optimized sequentially to refine one cumulative-logit prediction.
Deep supervision (Lee et al., 2015)	Joint optimization of intermediate and terminal classification losses	Replaces parallel auxiliary objectives with ordered cumulative-logit residual refinement.
Multi-exit networks (Teerapittayanon et al., 2016)	Confidence- or resource-based selection of an intermediate exit	Combines all heads into one cumulative prediction without early termination or confidence-based exit selection.
Boosted CNNs (Moghimi et al., 2016)	Multiple learners, boosting rounds, or instance reweighting	Performs logit-level residual refinement through lightweight heads within one shared backbone, without independent CNN learners or explicit sample reweighting.
AdaNet (Cortes et al., 2017)	Adaptive generation, selection, and weighting of subnetworks	Retains a predefined topology without architecture search or structural expansion, providing a fixed parameter count and predictable inference complexity.
Previous multi-head study (Yan et al., 2026)	Sequential residual heads attached to the regular stages of ResNet18	Maps the residual-correction stages to the heterogeneous structural transitions of Xception and evaluates this mapping against uniform-depth placement.

with the major structural transitions across the Stem, Entry, Middle, and Exit flows.

Signals Acquisition and Preprocessing. The process begins with the signals acquisition module, where target acoustic signals propagate through the complex ocean channel and are inevitably mixed with ambient marine noise. After array processing, the raw audio data is obtained. Subsequently, the data enters the feature extraction module. Through a standard pipeline consisting of framing, windowing, Fast Fourier Transform (FFT), Mel-scale filtering, and logarithmic compression, the 1D audio signals are effectively converted into 2D time-frequency Mel-spectrograms.

CNN Backbone with Hierarchical Stages. The generated spectrogram is fed into the CNN backbone. While we employ the Xception architecture as our primary feature extractor, the framework accommodates any network composed of cascaded convolutional blocks (Block 1 to Block K). As the data flows through these blocks, the network extracts hierarchical acoustic features at different depths (Stage 1 to Stage K), ranging from low-level spectral textures to high-level semantic abstractions.

Sequential Multi-Head Learning. Unlike traditional architectures with a single exit or parallel multi-task learning, our framework employs a step-wise sequential multi-head mechanism to progressively refine predictions. The first head, attached to Stage 1, produces the initial logit output $\mathbf{o}^{(1)}$, establishing the baseline prediction state $\mathbf{S}^{(1)} = \mathbf{o}^{(1)}$. For subsequent heads (Head 2 to K), we introduce an Additive Logit Update mechanism (indicated by the solid orange arrows). Crucially, during the forward pass, the accumulated state of the preceding stages ($\mathbf{S}^{(k-1)}$) is dynamically recomputed based on the evolving backbone features and the frozen weights of earlier heads. The cumulative prediction state $\mathbf{S}^{(k)}$ at stage k is then updated by adding the current head's residual prediction $\mathbf{o}^{(k)}$ to the previous state:

$$\mathbf{S}^{(k)} = \mathbf{S}^{(k-1)} + \mathbf{o}^{(k)}. \tag{1}$$

This structure ensures that deeper heads focus solely on the ambiguous patterns and missing information from the previous stages.

Residual Error Correction Strategy. The training process is guided by the Residual Error Correction path (indicated by blue dashed lines). The accumulated logits $\mathbf{S}^{(k)}$ are normalized via Softmax to generate the probability distribution $\mathbf{h}^{(k)}$, which is then compared against the ground truth $\mathbf{y}$. By minimizing this error ($\mathbf{y} - \mathbf{h}^{(k)}$), the network effectively forces the kth head to approximate the residual error left by the preceding ensemble, encouraging a sequential residual refinement process over the cumulative predictions.

Ensemble Output. During the inference phase, the final classification result is not dependent on a single branch. Instead, it is derived from the aggregation of outputs from all K heads ($\sum_{i=1}^{K} \mathbf{o}^{(i)}$), effectively leveraging the complementary, coarse-to-fine features learned across all hierarchical stages.

This architecture is modular and operates sequentially. By enforcing a progressive step-wise freezing protocol, each head is optimized within its corresponding training stage, while the trainable backbone region is gradually expanded from shallow to deep layers. At each stage, previously optimized heads are frozen as reference predictors, and the current head refines the accumulated prediction through residual correction. This coarse-to-fine refinement enhances robustness and improves generalization under complex underwater acoustic conditions, while reducing the optimization interference commonly observed in standard joint multi-head learning. Furthermore, the modular design ensures efficient training and inference. Because each newly activated stage focuses on correcting the residual errors left by earlier stages using a shared and progressively refined backbone, the framework achieves ensemble-like performance without incurring the prohibitive computational cost of conventional ensemble methods, making it well-suited for real-time deployment.

3.2. Xception backbone formulation

Let $D = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^{N}$ denote the training set, where $\mathbf{x}_i$ is the 2D Mel-spectrogram input and $\mathbf{y}_i \in \{0,1\}^C$ is the one-hot ground truth vector representing C distinct ship classes.

As illustrated in Fig. 2, the Xception backbone, denoted as $f_{\text{xc}}(\cdot;\boldsymbol{\theta}_{\text{xc}})$, is structurally organized into three hierarchical stages: the Entry flow, Middle flow, and Exit flow. By systematically replacing standard convolutions with depthwise separable convolutions, the architecture effectively decouples cross-channel correlations from spatial correlations, ensuring parameter efficiency.

Given an input spectrogram $\mathbf{x}_i$, the signal propagates through these hierarchical flows to generate the consolidated deep feature vector $\mathbf{z}_i$, which is subsequently mapped to the final output logits $\mathbf{o}_i$:

$$\mathbf{z}_i = f_{\text{xc}}(\mathbf{x}_i;\boldsymbol{\theta}_{\text{xc}}), \qquad \mathbf{o}_i = h_{\text{final}}(\mathbf{z}_i;\boldsymbol{\theta}_{\text{head}}), \tag{2}$$

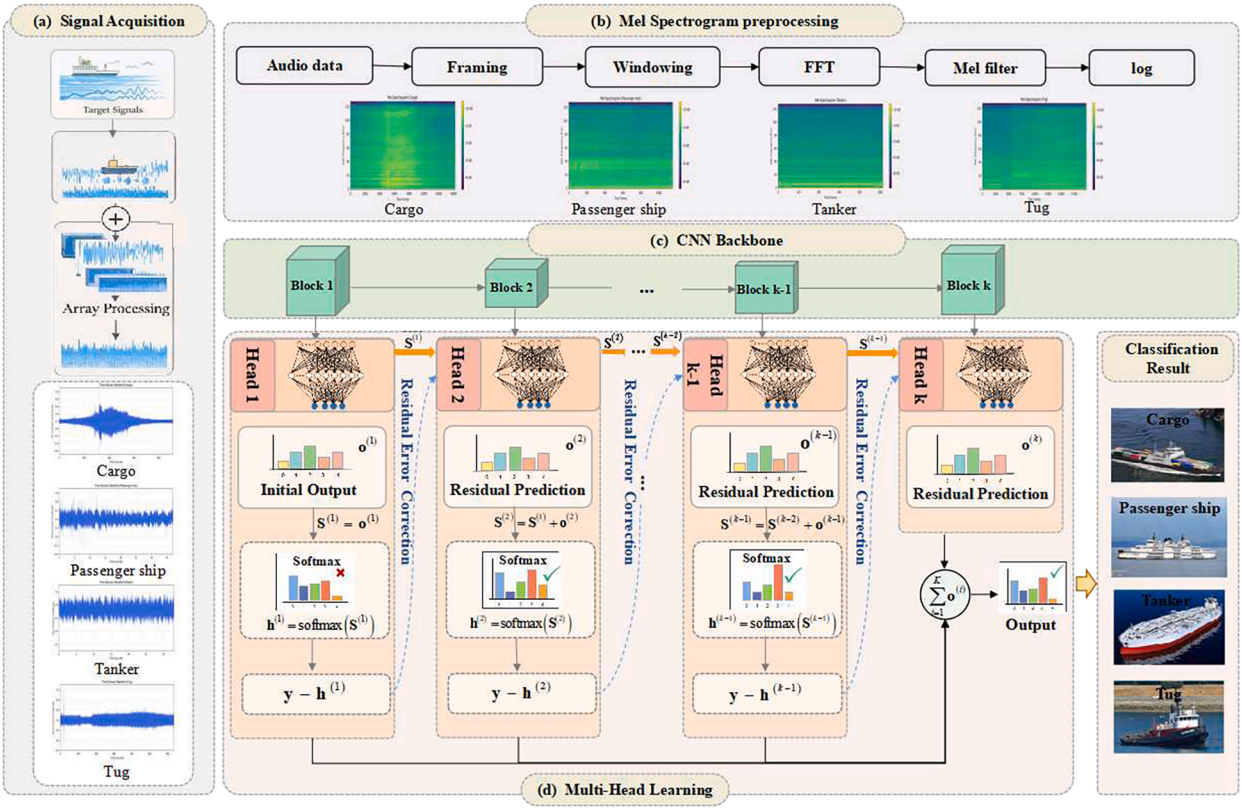


Fig. 1. The overall framework of the proposed STMH method. The system comprises Signals Acquisition, Mel-spectrogram preprocessing, a shared CNN Backbone, and a Multi-Head Learning module. The symbol $\mathbf{o}^{(k)}$ denotes the residual prediction of the kth head. The orange arrows indicate the additive logit update, resulting in the dynamically accumulated state $\mathbf{S}^{(k)} = \mathbf{S}^{(k-1)} + \mathbf{o}^{(k)}$. $\mathbf{h}^{(k)}$ represents the probability distribution derived via Softmax. The blue dashed lines illustrate the residual error correction mechanism ($\mathbf{y} - \mathbf{h}^{(k)}$), which drives the network to progressively compensate for prediction errors from preceding stages. Ultimately, the final classification decision is derived by aggregating the outputs across all stages, denoted as $\sum_{i=1}^{K} \mathbf{o}^{(i)}$.

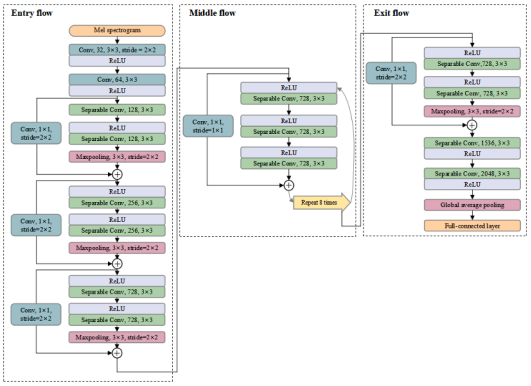


Fig. 2. Schematic of the Xception backbone. The architecture is modularized into Entry, Middle, and Exit flows, relying primarily on depthwise separable convolutions and residual connections for efficient, hierarchical feature extraction.

where h_{final} represents the final fully connected classification head parameterized by θ_{head}. For the baseline configuration, this single-head model is optimized using the standard cross-entropy loss.

3.3. Multi-head xception architecture

We augment the standard Xception architecture by attaching auxiliary classification heads at strategic points within its hierarchical flow structure. As depicted in Fig. 3, the K heads are distributed across varying network depths to capture acoustic features ranging from low-level spectral textures to high-level semantic abstractions.

3.3.1. Head attachment strategy

We select the attachment positions according to the major structural transitions of Xception. Since the Entry Flow progressively changes spatial resolution and channel depth, a head is placed after each Entry Flow block. In contrast, the Middle Flow consists of repeated blocks with a fixed feature scale, and therefore only one head is attached after its final block to avoid potentially redundant intermediate supervision.

Accordingly, to capture multi-scale acoustic features effectively, we integrate $K = 6$ lightweight heads at the following positions within the Xception backbone:

- Stem Head (h_1): Attached immediately after the initial convolutional stem. This head leverages high-resolution feature maps to extract coarse, low-level textural information prior to any residual learning.
- Entry Flow Heads (h_2, h_3, h_4): Positioned after each residual downsampling block within the Entry Flow. These heads monitor the feature transitions as the network progressively reduces spatial resolution while expanding channel depth.
- Middle Flow Head (h_5): Attached after the series of repeated blocks in the Middle Flow. Since this stage maintains a constant spatial resolution, h_5 focuses exclusively on deep feature refinement.
- Exit Flow Head (h_6): Serves as the final classifier at the network terminus, utilizing the most abstract semantic representations aggregated from the entire architecture.

Formally, let $h_k(\cdot)$ denote the kth auxiliary classifier head. To ensure the added branches remain efficient, each head is implemented as

a lightweight classification module consisting solely of a Global Average Pooling (GAP) layer followed by a single linear layer. Given an intermediate feature map $\mathbf{F}^{(k)} \in \mathbb{R}^{B\times C_k\times H_k\times W_k}$ extracted from the kth stage of the Xception backbone (where B is the batch size, C_k is the channel depth, and $H_k \times W_k$ represents the spatial resolution), the GAP operation first spatial-pools it into a feature vector in $\mathbb{R}^{B\times C_k}$. This vector is then mapped by the linear layer to the C-dimensional class logits:

$$\mathbf{o}^{(k)} = \mathbf{W}^{(k)}\,\mathrm{GAP}\big(\mathbf{F}^{(k)}\big) + \mathbf{b}^{(k)},$$

where $\mathbf{o}^{(k)} \in \mathbb{R}^{B\times C}$ denotes the output logits of the kth head, and $\mathbf{W}^{(k)}$ and $\mathbf{b}^{(k)}$ represent the learnable weights and biases, respectively. No additional convolutional, normalization, or hidden projection layers are introduced inside the head. For an input sample, the network produces a sequence of logit vectors $\{\mathbf{o}^{(1)}, \mathbf{o}^{(2)}, \ldots, \mathbf{o}^{(K)}\}$, where each $\mathbf{o}^{(k)} \in \mathbb{R}^{C}$. This minimalist design ensures that the integration of multiple heads introduces negligible computational overhead, preserving the real-time inference capability of the backbone.

3.3.2. Sequential residual learning

We employ a boosting-inspired training strategy in which each sequential head is optimized to correct the prediction errors remaining from its predecessors. Let $\mathbf{S}_i^{(k-1)}$ denote the accumulated logits produced by the first $k-1$ heads for sample i. Although the parameters of the preceding heads are frozen, their logits are dynamically recomputed at each forward pass using the current backbone features.

The kth head produces an output $\mathbf{o}_i^{(k)}$, which updates the cumulative prediction as

$$\mathbf{S}_i^{(k)} = \mathbf{S}_i^{(k-1)} + \mathbf{o}_i^{(k)}. \tag{3}$$

Let $\mathbf{h}_i^{(k)} = \mathrm{softmax}(\mathbf{S}_i^{(k)})$ denote the cumulative class-probability vector. Specifically, the probability assigned to class c is

$$h_{i,c}^{(k)} = \frac{\exp\left(S_{i,c}^{(k)}\right)}{\sum_{m=1}^{C}\exp\left(S_{i,m}^{(k)}\right)}. \tag{4}$$

The training objective at stage k is

$$\mathcal{L}_k = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{C} y_{i,c}\log h_{i,c}^{(k)} + \frac{\lambda}{2}\left\|\mathbf{\Theta}_{\text{active}}^{(k)}\right\|_2^2, \tag{5}$$

where λ controls the weight-decay strength, and $\mathbf{\Theta}_{\text{active}}^{(k)}$ denotes the parameters that remain trainable at stage k, including the active backbone layers and the current head.

To clarify the residual-learning mechanism, we derive the gradient with respect to the current head output. Let $\ell_i^{(k)}$ denote the cross-entropy loss of sample i at stage k. Since mini-batch averaging introduces only a positive scaling factor and does not change the gradient direction, we derive the logit-level gradient using the per-sample loss $\ell_i^{(k)}$. Substituting the softmax definition of $h_{i,c}^{(k)}$ into the cross-entropy loss and using $\sum_{c=1}^{C} y_{i,c} = 1$, the per-sample loss can be written in logit form as

$$\begin{aligned}\ell_i^{(k)} &= -\sum_{c=1}^{C} y_{i,c}\log h_{i,c}^{(k)} \\ &= -\mathbf{y}_i^T\mathbf{S}_i^{(k)} + \log\left(\sum_{m=1}^{C}\exp(S_{i,m}^{(k)})\right).\end{aligned} \tag{6}$$

Differentiating the first term with respect to $\mathbf{S}_i^{(k)}$ gives

$$\nabla_{\mathbf{S}_i^{(k)}}\left(-\mathbf{y}_i^T\mathbf{S}_i^{(k)}\right) = -\mathbf{y}_i. \tag{7}$$

The gradient of the log-sum-exp term is

$$\nabla_{\mathbf{S}_i^{(k)}}\log\left(\sum_{m=1}^{C}\exp(S_{i,m}^{(k)})\right) = \mathrm{softmax}(\mathbf{S}_i^{(k)}) = \mathbf{h}_i^{(k)}. \tag{8}$$

Combining the two terms yields

$$\nabla_{\mathbf{S}_i^{(k)}}\ell_i^{(k)} = \mathbf{h}_i^{(k)} - \mathbf{y}_i. \tag{9}$$

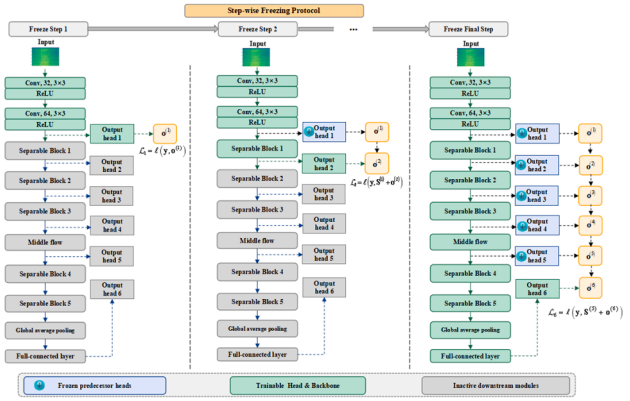


Fig. 3. Sequential Multi-head Training with Step-wise Freezing Protocol. Green modules denote the trainable backbone prefix and the active output head at the current stage. Blue modules denote previously optimized and frozen components. Gray modules denote deeper backbone blocks or future heads that are inactive in the current stage. The trainable region progressively expands from shallow to deep stages until the full backbone is activated at the final step.

According to $\mathbf{S}_i^{(k)} = \mathbf{S}_i^{(k-1)} + \mathbf{o}_i^{(k)}$, the accumulated logits of the preceding heads have no direct dependence on the current head output. Therefore, $\partial\mathbf{S}_i^{(k)}/\partial\mathbf{o}_i^{(k)} = \mathbf{I}$. Applying the chain rule gives

$$\begin{aligned}\nabla_{\mathbf{o}_i^{(k)}}\ell_i^{(k)} &= \underbrace{\left(\frac{\partial\mathbf{S}_i^{(k)}}{\partial\mathbf{o}_i^{(k)}}\right)^T}_{=\mathbf{I}}\nabla_{\mathbf{S}_i^{(k)}}\ell_i^{(k)} \\ &= \mathbf{h}_i^{(k)} - \mathbf{y}_i.\end{aligned} \tag{10}$$

The L_2 term does not directly depend on the logits and therefore does not alter this logit-level derivative. Consequently, the negative-gradient direction is

$$-\nabla_{\mathbf{o}_i^{(k)}}\ell_i^{(k)} = \mathbf{y}_i - \mathbf{h}_i^{(k)}. \tag{11}$$

This direction represents the remaining prediction discrepancy of the cumulative classifier. With the parameters of the preceding heads frozen, optimizing the stage-k cumulative objective encourages the newly introduced head to correct the prediction errors remaining after the preceding stages. STMH thus realizes a boosting-inspired sequential residual correction process within a single shared network.

3.3.3. Step-wise freezing protocol

To reconcile sequential error correction with stable stage-wise optimization, we adopt a progressive step-wise freezing protocol, as illustrated in Fig. 3. The figure presents three representative stages of training: the initial stage (Freeze Step 1), an intermediate refinement stage (Freeze Step 2), and the final refinement stage (Freeze Final Step). In each stage, green modules denote the trainable components involved in the current optimization step, blue modules denote previously optimized and frozen predecessor heads, and gray modules denote downstream backbone blocks or future heads that remain inactive at the current stage.

At the first stage, only the shallow backbone prefix associated with the first output head (h_1) and the head itself are optimized. This stage establishes the initial prediction $\mathbf{o}^{(1)}$ and learns a coarse feature representation from the input spectrogram. As shown in the left panel of Fig. 3, only the first head participates in supervision, while all deeper backbone blocks and the remaining heads are inactive.

From the second stage onward, the trainable region is progressively extended deeper into the network. As illustrated in the middle panel of Fig. 3, the parameters of previously trained heads are frozen, while the backbone prefix up to the attachment point of the current head and the current head itself remain trainable. Importantly, freezing earlier

Algorithm 1 STMH training with progressive step-wise freezing.

Input: Training data D, total heads K, epochs per stage E_k, backbone θ_{xc}.
for stage $k = 1$ to K **do**
  Freeze predecessor heads $h_1, \ldots, h_{k-1}$.
  Activate the backbone prefix up to h_k and the current head h_k.
  for epoch $= 1$ to E_k **do**
    for each batch $(\mathbf{x}, \mathbf{y}) \in D$ **do**
      Extract stage-wise features using the active backbone prefix.
      if $k > 1$ **then**
        Recompute predecessor logits: $\mathbf{S}^{(k-1)} = \sum_{j=1}^{k-1} h_j(\mathbf{x})$.
      else
        Set $\mathbf{S}^{(0)} = \mathbf{0}$.
      end if
      Compute current output $\mathbf{o}^{(k)} = h_k(\mathbf{x})$.
      Update cumulative logits: $\mathbf{S}^{(k)} = \mathbf{S}^{(k-1)} + \mathbf{o}^{(k)}$.
      Update active parameters $\boldsymbol{\Theta}^{(k)}_{\text{active}}$ by minimizing $\mathcal{L}_{\text{CE}}(\mathbf{S}^{(k)}, \mathbf{y}) + \frac{\lambda}{2}\|\boldsymbol{\Theta}^{(k)}_{\text{active}}\|_2^2$ with Adam.
    end for
  end for
end for
Output: $\mathbf{h}^{(K)} = \text{softmax}\left(\sum_{k=1}^{K} \mathbf{o}^{(k)}\right)$.

heads does not imply that their outputs are treated as fixed constants. Instead, their logits are dynamically recomputed under the currently updated backbone features at each forward pass, thereby providing stage-wise reference predictions for the active head. The current head is then trained to refine the accumulated prediction state, for example,

$$\mathcal{L}_2 = \ell\left(\mathbf{y}, \mathbf{S}^{(1)} + \mathbf{o}^{(2)}\right).$$

In this way, earlier heads serve as parameter-fixed reference predictors, while the newly activated backbone region and current head cooperatively learn to correct the residual errors left by previous stages.

This process continues until the final stage, shown in the right panel of Fig. 3, where the trainable region has expanded to cover the full backbone and only the last head remains active. The final head refines the cumulative prediction built by all previous stages, yielding

$$\mathcal{L}_K = \ell\left(\mathbf{y}, \mathbf{S}^{(K-1)} + \mathbf{o}^{(K)}\right).$$

Therefore, the overall training procedure follows a coarse-to-fine progression: shallow heads first learn coarse acoustic cues, while deeper heads and progressively deeper backbone blocks are introduced later to focus on the remaining ambiguous patterns that earlier stages fail to resolve.

The proposed protocol is motivated by two complementary considerations. First, freezing previously optimized heads stabilizes earlier predictors and reduces the inter-head optimization interference commonly observed in joint multi-head training. Second, progressively expanding the trainable backbone depth preserves feature plasticity while preventing deeper layers from being prematurely optimized before sufficiently informative intermediate representations have been established. As a result, the proposed protocol enables sequential residual refinement without locking the model into an overly shallow feature space.

4. Experimental setup

4.1. Datasets

To comprehensively evaluate the scalability and robustness of the proposed STMH-Xception framework, we utilize two public underwater ship-radiated noise datasets of significantly different scales: ShipsEar and DeepShip.

ShipsEar represents a challenging data-scarce scenario. Captured along the Atlantic coast of Spain between 2012 and 2013, it consists of 90 WAV-format recordings sampled at 52,734 Hz. The durations vary drastically from 15 s to 10 min, yielding approximately three hours of audio in total. The dataset includes 11 specific vessel types and one natural background noise category, which are consolidated into five main classes to align with established UATR benchmarks, as detailed in Table 2.

Table 2
Target category in the ShipsEar dataset.

Category	Targets
Class-A	fishing boats, trawlers, mussel boats, tugboats, dredgers
Class-B	motorboats, pilot boats, sailboats
Class-C	passenger liners
Class-D	ocean liners, no-vessels
Class-E	ocean noise

DeepShip serves as the large-scale benchmark in our study, representing long-duration, continuous marine environments. It comprises 613 real-world underwater acoustic recordings acquired with a sampling rate of 32 kHz. The total duration spans 47.07 h, with individual recordings ranging from 6 s to 25 min. The dataset encompasses 265 distinct vessels categorized into four commercial classes: Cargo, Passenger, Tanker, and Tug.

4.2. Data partitioning and preprocessing

As highlighted by recent studies (Bae et al., 2025), the choice of dataset partitioning strategy in UATR—such as random split, sequential split, or recording-ID-based split—significantly impacts the reliability of generalization evaluations. To ensure a deployment-relevant assessment, we strictly adopt the most stringent recording-ID-based split protocol, consistent with our previous work (Yan et al., 2026). Under this setting, the training and test sets are completely disjoint at the recording level, guaranteeing that no data leakage occurs across contiguous temporal frames. This protocol authentically simulates a real-world scenario where models must generalize to entirely unseen environmental conditions and vessel acoustic signatures.

Specifically, for the DeepShip dataset, 15% of the recordings for each class are strictly held out for testing, while the remaining 85% form the training pool. For the data-scarce ShipsEar dataset, we adopt a carefully predefined recording-level split that maintains an approximate 8:2 train-test ratio while preserving class balance. The exact recording IDs used for the training and testing partitions for both datasets strictly follow the configurations detailed in Yan et al. (2026). Finally, for both datasets, a portion of the training clips (15% for ShipsEar and 20% for DeepShip) are explicitly reserved as a validation set for model selection and to monitor convergence.

4.3. Data preprocessing

Raw acoustic recordings are first loaded as mono signals and resampled to a unified rate of $f_s = 16$ kHz. The waveforms are then segmented into fixed-duration clips of $T_{\text{sec}} = 3$ s, yielding $L = f_s T_{\text{sec}} = 48{,}000$ samples per clip. To ensure numerical consistency across diverse recording conditions and varying equipment sensitivities, each segmented waveform is normalized to a zero-mean and unit-variance distribution prior to the time-frequency transformation.

Subsequently, acoustic feature extraction is performed via the Short-Time Fourier Transform (STFT) using a Hann window with a length of $N_{\text{win}} = N_{\text{fft}} = 4096$ samples and a hop size of $H = 1024$ samples. For each time frame τ and frequency bin k, the complex spectrum $X(k, \tau)$ is obtained and converted into a power spectrogram:

$$S(k, \tau) = \left|X(k, \tau)\right|^2. \tag{12}$$

Finally, the power spectrogram $S(k, \tau)$ is projected onto a bank of $F_{\text{mel}} = 128$ Mel filters, yielding the final compact input representation $M \in \mathbb{R}^{128 \times T}$.

4.4. Implementation details

The proposed framework is implemented in PyTorch 1.13 and executed on a workstation equipped with an NVIDIA GeForce RTX 4090 Ti GPU and an Intel Xeon Silver 4310 CPU. Network optimization is performed using the Adam optimizer with a batch size of 64, an initial learning rate of 1×10^{-4}, and a weight decay coefficient of 1×10^{-4}.

The STMH framework is trained utilizing a step-wise curriculum across $K = 6$ successive stages, with each stage allocated a maximum of 10 epochs. During the kth stage, the cumulative prediction state is updated via $S^{(k)} = \sum_{i=1}^{k} o^{(i)}$.

Throughout the training process, the learning rate is modulated by a cosine annealing scheduler. To prevent overfitting, an early stopping mechanism is triggered if the validation loss fails to improve for 5 consecutive epochs. Finally, to ensure statistical reliability while maintaining computational feasibility, the number of independent runs is tailored to the data scale. For the ShipsEar dataset, experiments are repeated 10 times using distinct random seeds to account for initialization variance. For the larger DeepShip dataset, where the extensive volume of data naturally stabilizes the optimization trajectory, 5 independent runs are conducted. We report the resulting mean and standard deviation for all evaluated metrics.

4.5. Evaluation metrics

Model performance is assessed using four standard indicators: Accuracy, Precision, Recall, and F1-score. Furthermore, to provide a holistic evaluation, we define a composite *Average* score as the arithmetic mean of these metrics:

$$\text{Average} = \frac{1}{4}(\text{Acc} + \text{Pre} + \text{Rec} + \text{F1}). \tag{13}$$

The standard metrics are defined as follows, where TP, TN, FP, and FN represent true positives, true negatives, false positives, and false negatives, respectively:

$$\text{Acc} = \frac{\text{TP} + \text{TN}}{\text{Total}}, \quad \text{Pre} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \tag{14}$$

$$\text{Rec} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad \text{F1} = \frac{2 \cdot \text{Pre} \cdot \text{Rec}}{\text{Pre} + \text{Rec}}. \tag{15}$$

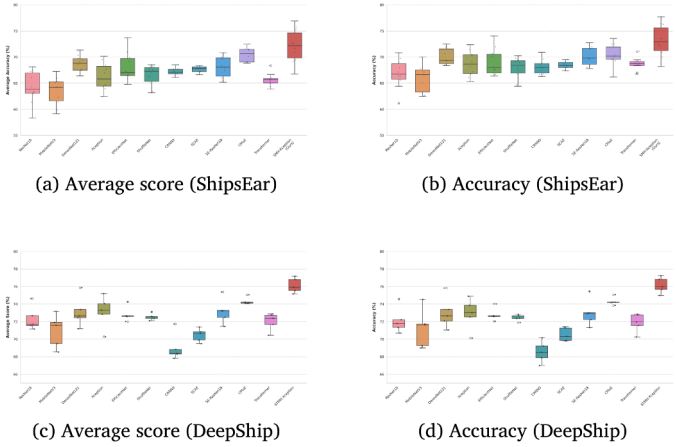
5. Results and analysis

5.1. Comparative study

We benchmark the proposed STMH-Xception against a comprehensive suite of eleven state-of-the-art baselines. These include lightweight CNNs (MobileNetV3 Akbarian & Sedaaghi, 2025, ShuffleNetV2 Zhang et al., 2018a), standard high-capacity architectures (ResNet18 Hong et al., 2021, DenseNet121 Doan et al., 2020), and advanced specialized models (CMoE Xie et al., 2024, Transformers Feng & Zhu, 2022). The results across both ShipsEar and DeepShip datasets are detailed in Table 3.

On the ShipsEar dataset, data scarcity and class imbalance pose significant challenges. While baselines like CMoE and SE-ResNet18 reach average scores of around 70%, the vision Transformer degrades to 65.65%, likely due to its lack of inductive bias when training data is limited. In contrast, STMH-Xception achieves an accuracy of 73.10% and an average score of 72.13%. This outperforms the Xception baseline by +4.38 percentage points (pp) and the strongest compared model CMoE by +2.69 pp in accuracy, demonstrating that the proposed STMH strategy improves recognition performance under data-scarce conditions through step-wise residual correction.

The DeepShip dataset presents continuous, long-duration recordings with intra-class variations and non-stationary background noise. Benefiting from the abundance of training data, most baselines exhibit higher overall accuracies here compared to ShipsEar. For instance, CMoE and DenseNet121 achieve average scores of 74.32% and 73.10%, respectively. Furthermore, STMH-Xception demonstrates robust scalability, reaching an accuracy of 76.14% and an average score of 76.12%. It improves upon Xception by +3.25 pp and outperforms the strongest compared model CMoE by +1.83 pp in accuracy, indicating its capability to extract more discriminative acoustic features in continuous marine environments.



(a) Average score (ShipsEar) (b) Accuracy (ShipsEar)

(c) Average score (DeepShip) (d) Accuracy (DeepShip)

Fig. 4. Comprehensive analysis of performance distributions across data scales and metrics. (a) and (b) represent Average score and Accuracy on the ShipsEar dataset. (c) and (d) represent Average score and Accuracy on the DeepShip dataset.

Beyond the numerical ranking, these improvements are practically meaningful for continuous underwater acoustic monitoring. In passive sonar or AUV-based monitoring systems, classifiers are often required to process long-duration acoustic streams, where even a few percentage points of accuracy improvement can accumulate into a substantial reduction in classification errors and vessel-type misclassifications. This is particularly important for commercial vessel categories with similar radiated-noise patterns and overlapping time-frequency characteristics.

To examine the run-wise performance distributions of the evaluated methods, Fig. 4 presents boxplots of the Accuracy and Average scores obtained across multiple independent runs. On the ShipsEar dataset (Fig. 4(a) and (b)), several models exhibit noticeable run-to-run variability, which is expected given the limited dataset size and the sensitivity of model training to random initialization and data sampling. STMH-Xception shifts the overall performance distribution toward a higher range and achieves the highest median performance among the evaluated methods. However, its run-to-run variability remains non-negligible; therefore, the boxplots are interpreted primarily as evidence of improved central performance rather than uniformly reduced variance. On the DeepShip dataset (Fig. 4(c) and (d)), the larger amount of training data generally results in narrower performance distributions across the evaluated methods. STMH-Xception maintains the highest median and remains concentrated within the leading performance range. Overall, the boxplots indicate that the proposed method achieves consistently higher run-wise performance across both datasets.

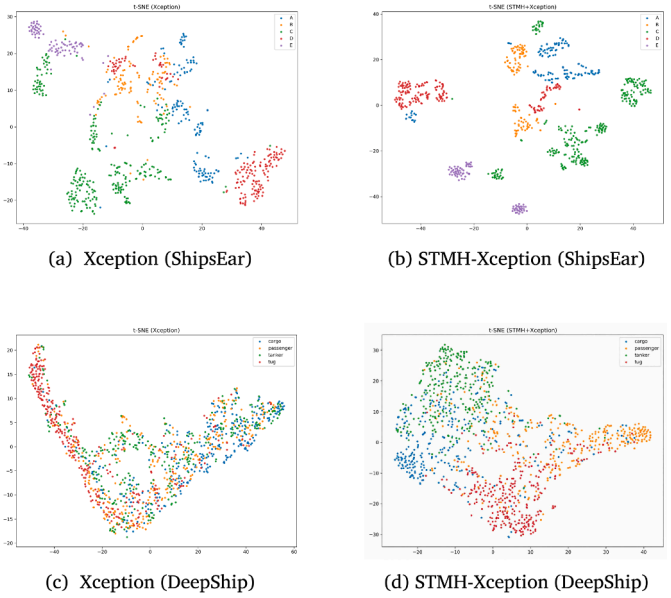
5.2. Visualization and analysis

To visually examine the separability of the learned representations, we extract the penultimate-layer embeddings from both the ShipsEar and DeepShip test sets and project them into a 2D space using t-SNE. Fig. 5 compares the feature distributions generated by the baseline Xception against our proposed STMH-Xception.

As shown in Fig. 5, the baseline embeddings (Fig. 5a) exhibit notable overlap and dispersion on the ShipsEar dataset. Specifically, Classes B and D are largely overlapped, and Class C appears split into multiple fragmented sub-clusters, indicating limited intra-class compactness and insufficient inter-class separation under data-scarce conditions. In con-

Table 3
Comprehensive performance comparison on ShipsEar and DeepShip datasets. Best results are in bold.

Model	ShipsEar Dataset (%)					DeepShip Dataset (%)				
	Acc ↑	Precision ↑	Recall ↑	F1-score ↑	Average ↑	Acc ↑	Precision ↑	Recall ↑	F1-score ↑	Average ↑
ResNet18 (Hong et al., 2021)	66.84 ± 2.80	63.44 ± 3.53	63.60 ± 2.95	63.05 ± 3.14	64.23 ± 3.05	72.12 ± 1.48	73.03 ± 1.26	72.00 ± 1.48	72.20 ± 1.44	72.34 ± 1.40
MobileNetV3 (Akbarian & Sedaghi, 2025)	65.87 ± 2.58	64.34 ± 2.71	61.40 ± 3.09	62.21 ± 2.83	63.46 ± 2.69	71.24 ± 2.25	71.21 ± 1.92	70.61 ± 1.79	70.70 ± 1.72	70.94 ± 1.88
ShuffleNetV2 (Zhang et al., 2018a)	67.85 ± 2.05	67.30 ± 2.33	65.16 ± 2.02	65.60 ± 2.05	66.48 ± 1.98	72.43 ± 0.35	72.89 ± 0.50	72.45 ± 0.34	72.40 ± 0.38	72.54 ± 0.37
EfficientNet (Tan & Le, 2019)	68.99 ± 2.59	67.99 ± 2.94	68.36 ± 2.77	67.20 ± 3.02	68.14 ± 2.74	72.79 ± 0.74	73.08 ± 1.04	72.71 ± 0.81	72.71 ± 0.79	72.82 ± 0.84
DenseNet121 (Doan et al., 2020)	70.04 ± 1.60	70.04 ± 1.24	66.99 ± 2.40	67.53 ± 2.17	68.65 ± 1.65	73.00 ± 1.80	73.48 ± 1.68	72.89 ± 1.76	73.03 ± 1.75	73.10 ± 1.74
SE-ResNet18 (Chen et al., 2025)	70.13 ± 1.78	67.98 ± 2.02	67.08 ± 2.23	67.01 ± 2.17	68.05 ± 2.03	72.99 ± 1.53	73.64 ± 1.38	73.01 ± 1.47	73.01 ± 1.44	73.16 ± 1.44
CRNN9 (Liu et al., 2021)	68.10 ± 1.45	67.26 ± 1.33	66.35 ± 1.58	66.98 ± 1.23	67.17 ± 0.71	68.55 ± 1.21	69.13 ± 2.06	69.13 ± 2.13	69.22 ± 2.00	69.01 ± 1.56
SCAE (Irfan et al., 2021)	68.43 ± 0.77	67.20 ± 0.99	66.45 ± 1.23	68.58 ± 0.87	67.67 ± 0.57	70.53 ± 0.77	70.40 ± 0.76	70.46 ± 0.75	70.42 ± 0.85	70.45 ± 0.75
CMoE (Xie et al., 2024)	70.41 ± 2.08	70.85 ± 2.02	69.30 ± 2.21	69.55 ± 1.98	70.42 ± 1.36	74.31 ± 0.45	74.59 ± 0.37	74.13 ± 0.48	74.24 ± 0.44	74.32 ± 0.42
Transformer (Feng & Zhu, 2022)	68.66 ± 1.22	64.56 ± 1.38	65.08 ± 1.28	64.30 ± 1.30	65.65 ± 1.25	71.87 ± 1.06	72.30 ± 0.94	71.89 ± 1.01	72.02 ± 1.06	72.02 ± 0.99
Xception(Chollet, 2017)	68.72 ± 2.39	64.73 ± 3.03	66.39 ± 2.39	65.23 ± 2.77	66.27 ± 2.62	72.89 ± 1.79	73.72 ± 1.86	72.92 ± 1.84	73.01 ± 1.82	73.13 ± 1.82
STMH-Xception	**73.10 ± 3.03**	**71.89 ± 3.31**	**72.02 ± 3.23**	**71.53 ± 3.33**	**72.13 ± 3.21**	**76.14 ± 0.90**	**76.26 ± 0.72**	**76.03 ± 0.92**	**76.06 ± 0.88**	**76.12 ± 0.84**



(a) Xception (ShipsEar) (b) STMH-Xception (ShipsEar)

(c) Xception (DeepShip) (d) STMH-Xception (DeepShip)

Fig. 5. t-SNE visualization of feature embeddings, comparing the baseline Xception with our proposed STMH-Xception model across different datasets.

trast, the embeddings optimized by STMH-Xception (Fig. 5b) present better-defined cluster boundaries. The previously overlapped Class B and D samples become distinctly separable, and several categories (e.g., Class E) form more compact, cohesive groups.

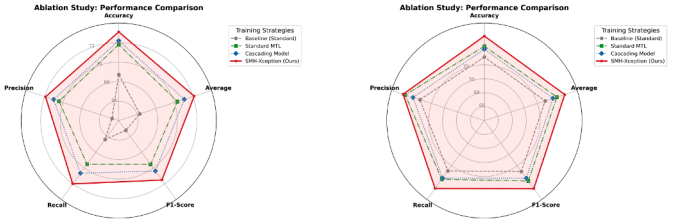
The bottom row illustrates the feature space for the DeepShip dataset. Here, the intra-class variance caused by varying vessel speeds and non-stationary marine noise poses a different challenge. The baseline model (Fig. 5c) produces a less distinct feature space, where acoustically similar commercial vessels (such as Cargo and Passenger classes) intermingle at the boundaries. By applying the STMH strategy (Fig. 5d), the network reduces these ambiguities. The clusters become more concentrated, and the margin between different vessel classes is increased.

These dual-dataset observations visually corroborate the quantitative results. By forcing subsequent auxiliary heads to correct the residual errors of earlier stages, the proposed sequential supervision minimizes ambiguous feature representations. This process drives the network to learn a structured and robust feature organization, regardless of the underlying dataset scale. From a practical perspective, the improved t-SNE separability further suggests that STMH-Xception learns more discriminative acoustic representations for vessel classes with similar spectral patterns. The enhanced intra-class compactness and inter-class margins can reduce ambiguous decisions among acoustically similar vessels, thereby supporting more reliable recognition in operational monitoring scenarios.

5.3. *Effectiveness and strategy analysis of STMH*

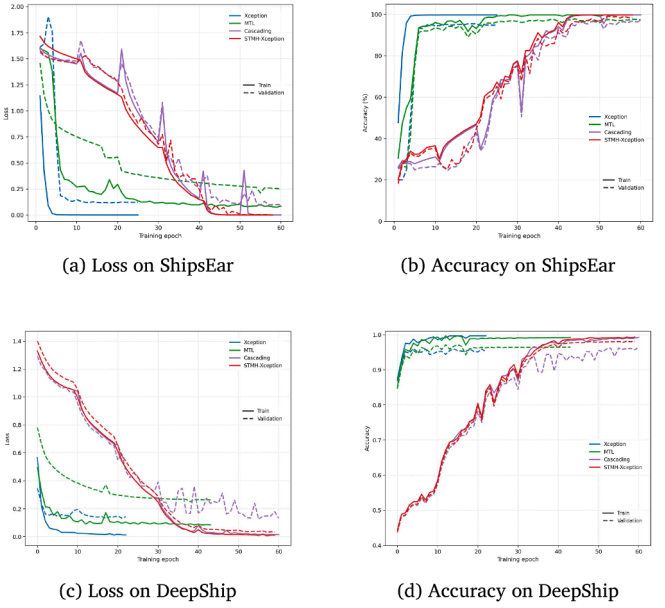
To comprehensively assess the contribution of the proposed STMH strategy, we conduct an ablation study comparing our framework against three representative training paradigms: (i) a single-head baseline (Standard Backbone), (ii) standard MTL with simultaneous joint supervision of all auxiliary heads, and (iii) a Cascading model that progressively reuses intermediate representations without explicit residual targeting. Fig. 6 presents a radar-chart comparison of these four strategies across all five evaluation metrics on both the ShipsEar and DeepShip datasets.

All multi-head variants consistently outperform the single-head baseline across both data scales, verifying that introducing intermediate supervision is beneficial for deep acoustic feature extraction. However, the exact optimization strategy heavily dictates the performance ceiling.



(a) ShipsEar (b) DeepShip

Fig. 6. Radar-chart comparison of different training strategies. The proposed STMH strategy (red solid line) consistently envelops the other configurations on both the (a) ShipsEar and (b) DeepShip datasets.



(a) Loss on ShipsEar (b) Accuracy on ShipsEar

(c) Loss on DeepShip (d) Accuracy on DeepShip

Fig. 7. Training and validation performance trends on the ShipsEar and DeepShip datasets. Solid lines denote training performance, while dashed lines denote validation performance.

As depicted in Fig. 6(a), standard MTL improves the baseline accuracy from 68.72% to 71.79%, but its metric polygon remains strictly contained within that of the STMH strategy. This performance gap highlights a common vulnerability of MTL, optimizing multiple heads simultaneously on a limited dataset often leads to gradient conflicts, where shallow and deep heads pull the shared backbone in diverging directions, which can exacerbate overfitting. The cascading model mitigates this slightly, raising the accuracy to 72.20% through progressive feature reuse. Nevertheless, STMH-Xception achieves the highest accuracy of 73.10% and the most balanced overall metric profile (average score increasing from 66.27% to 72.13%), indicating that sequential stage-wise freezing provides effective regularization against data scarcity.

The behavioral differences between these strategies exhibit distinct patterns on the DeepShip dataset (Fig. 6(b)). Benefiting from abundant data, MTL (74.68%) outpaces the cascading model (74.24%), suggesting that joint optimization is more viable when the risk of overfitting is low. However, both strategies still struggle to break the performance saturation of the baseline model (73.13%), yielding only moderate improvements of around 1.1% $\sim$ 1.5%. In contrast, the proposed STMH strategy (red solid line) exhibits a significant outward expansion across all five axes, elevating the accuracy to 76.14% and the average score to 76.12%. This demonstrates that STMH effectively overcomes the limitations of standard joint or sequential training.

The fundamental advantage of STMH lies in its boosting-inspired design. Unlike MTL's joint optimization or cascading's passive feature reuse, STMH explicitly formulates each stage as a residual refinement. By freezing earlier heads, the network is forced to focus its active learning capacity exclusively on the remaining ambiguous samples that previous stages failed to classify. This "coarse-to-fine" sequential error correction serves as an effective optimization strategy in our experiments, yielding consistent performance improvements across the evaluated datasets.

In addition to comparing different training paradigms, we examine the effect of auxiliary-head placement on STMH-Xception. The uniform-depth configuration distributes six identical GAP-linear heads at approximately equal depth intervals. However, Xception is structurally heterogeneous, and equal depth intervals do not necessarily correspond to distinct representation levels. The Stem extracts low-level time–frequency patterns, while the three Entry Flow blocks progressively change feature resolution and channel depth. Therefore, a head is placed after the Stem and after each Entry Flow block to provide supervision at these major feature transitions. In contrast, the Middle Flow consists of repeated blocks operating at a fixed feature scale; only one head is attached after its final block to avoid potentially redundant supervision at structurally similar representations. The final head is placed after the Exit Flow to capture the highest-level semantic representation. Consequently, the proposed architecture-aware configuration places supervision more densely around major structural transitions and more sparsely within the repetitive Middle Flow.

As shown in Table 4, uniform-depth placement already improves the mean performance of Xception, demonstrating the general effect of sequential intermediate supervision. With the same model capacity, architecture-aware placement provides additional mean Accuracy/Average gains of +0.83/ + 0.89 percentage points on ShipsEar and +0.65/ + 0.61 percentage points on DeepShip. These results provide supporting evidence for selecting attachment positions according to Xception's major structural transitions.

5.4. Generalization and robustness analysis

Although the comparative results demonstrate the accuracy advantage of STMH-Xception, final test accuracy alone is insufficient to determine whether the proposed method improves generalization. We evaluate the proposed framework from three complementary perspectives: training–validation performance trends, robustness to unseen colored-noise perturbations, and controlled comparisons with conventional anti-overfitting techniques.

Fig. 7 compares the training and validation curves of Xception, joint MTL, Cascading, and STMH-Xception on both datasets. STMH-Xception shows no pronounced late-stage training–validation divergence: its validation loss continues to decrease, while its validation accuracy gradually approaches the training accuracy. Its relatively gradual convergence is expected because STMH-Xception and Cascading concatenate successive head-training stages, whereas Xception and joint MTL optimize their classifiers from the outset; activating a new stage may also cause temporary fluctuations. Unlike Cascading, STMH-Xception uses cumulative-logit residual correction to direct each successive head toward the remaining prediction errors. Overall, its accuracy improvement is accompanied by competitive validation convergence rather than merely increased fitting of the training data.

We further evaluate robustness by adding colored red noise directly to the test waveforms. Following the underwater-noise spectral characteristics discussed in Hazelwood and Connelly (2005), red noise is used as a controlled frequency-dependent perturbation rather than spectrally flat Gaussian white noise. The tested SNR levels are 30, 20, 10, 5, 0, −5, and −10 dB. All trained checkpoints remain fixed without retraining or fine-tuning.

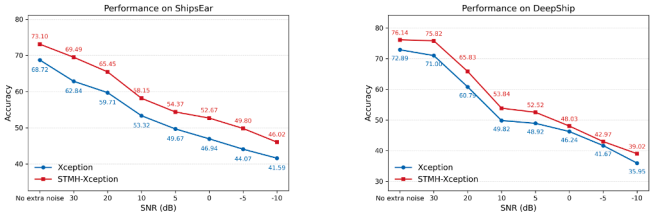
As shown in Fig. 8, STMH-Xception maintains higher mean accuracy than Xception at every evaluated SNR level. Across the seven noisy conditions, its mean advantages are 5.40 percentage points on ShipsEar

Table 4
Effect of auxiliary-head placement on Xception.

Placement	ShipsEar (%)		DeepShip (%)	
	Acc.	Avg.	Acc.	Avg.
Xception	68.72 ± 2.39	66.27 ± 2.62	72.89 ± 1.79	73.13 ± 1.82
Uniform-depth	72.27 ± 2.26	71.24 ± 2.51	75.49 ± 0.87	75.51 ± 0.88
Architecture-aware (Ours)	**73.10 ± 3.03**	**72.13 ± 3.21**	**76.14 ± 0.90**	**76.12 ± 0.84**

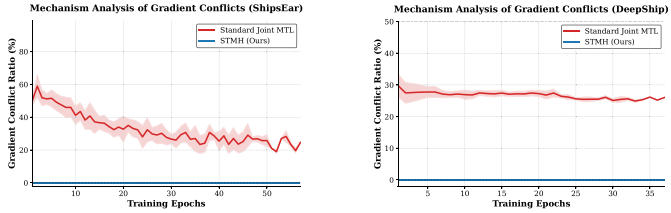
Table 5
Comparison with conventional anti-overfitting techniques on ShipsEar and DeepShip datasets. Best results are in bold.

Method	ShipsEar Dataset (%)					DeepShip Dataset (%)				
	Acc ↑	Precision ↑	Recall ↑	F1-score ↑	Average ↑	Acc ↑	Precision ↑	Recall ↑	F1-score ↑	Average ↑
Xception	68.72 ± 2.39	64.73 ± 3.03	66.39 ± 2.39	65.25 ± 2.77	66.27 ± 2.62	72.89 ± 1.79	73.72 ± 1.86	72.92 ± 1.84	73.01 ± 1.82	73.13 ± 1.82
Xception + Dropout	69.11 ± 1.93	65.22 ± 2.57	66.49 ± 2.41	65.53 ± 2.45	66.59 ± 2.31	73.22 ± 1.79	74.00 ± 1.55	73.20 ± 1.75	73.32 ± 1.75	73.43 ± 1.71
Xception + Label Smoothing	69.46 ± 2.47	65.99 ± 3.24	66.27 ± 2.33	65.18 ± 2.86	66.73 ± 2.69	73.76 ± 0.55	74.41 ± 0.52	73.74 ± 0.60	73.84 ± 0.60	73.94 ± 0.57
Xception + Mixup	68.54 ± 2.12	65.17 ± 3.38	64.25 ± 2.53	62.91 ± 2.73	64.97 ± 2.40	73.97 ± 0.92	74.38 ± 0.87	73.95 ± 0.88	73.95 ± 0.94	74.06 ± 0.90
Xception + SpecAugment	69.99 ± 2.12	67.78 ± 2.34	66.71 ± 2.02	66.72 ± 2.05	67.80 ± 2.06	73.27 ± 1.35	74.04 ± 0.67	73.22 ± 1.35	73.29 ± 1.28	73.45 ± 1.13
STMH-Xception	**73.10 ± 3.03**	**71.89 ± 3.31**	**72.02 ± 3.23**	**71.53 ± 3.33**	**72.13 ± 3.21**	**76.14 ± 0.90**	**76.26 ± 0.72**	**76.03 ± 0.92**	**76.06 ± 0.88**	**76.12 ± 0.84**



(a) ShipsEar (b) DeepShip

Fig. 8. Robustness comparison under different SNR conditions on ShipsEar and DeepShip.



(a) ShipsEar (b) DeepShip

Fig. 9. Analysis of gradient conflict ratios across two datasets. (a) and (b) represent the optimization trajectories on the ShipsEar dataset and DeepShip dataset, respectively. The solid lines indicate the mean conflict ratio, while the shaded regions denote the standard deviation.

and 3.38 percentage points on DeepShip. These results provide the evidence that STMH-Xception is less sensitive to perturbations and that its performance improvement is not restricted to the original clean test condition.

Finally, we compare STMH-Xception with Dropout, Label Smoothing, Mixup, and SpecAugment under the same Xception backbone and evaluation protocol. As reported in Table 5, STMH-Xception achieves the highest mean Accuracy, Precision, Recall, F1-score, and Average score on both datasets.

Compared with the strongest conventional method, STMH-Xception improves the Accuracy/Average score by 3.11/4.33 percentage points over SpecAugment on ShipsEar and by 2.17/2.06 percentage points over Mixup on DeepShip. Conventional techniques regularize activations, targets, or training samples, whereas STMH-Xception changes the supervision structure by sequentially correcting residual prediction errors. Its improvement therefore cannot be explained merely by adjusting the strength of a conventional regularizer.

Taken together, the validation trends, robustness results under unseen colored-noise perturbations, and comparisons with conventional anti-overfitting techniques provide complementary evidence that the performance gain of STMH-Xception is accompanied by improved generalization behavior.

5.5. Mechanism analysis of gradient conflicts

To investigate how the proposed STMH strategy mitigates optimization interference compared to standard Multi-Task Learning (MTL), we quantitatively analyze the gradient conflict ratios during the training process. In standard MTL, auxiliary heads often capture diverse, scale-specific acoustic patterns, which can induce divergent optimization directions for the shared backbone. We evaluate this phenomenon using the Xception backbone by computing the cosine similarity between the gradients of the ith and jth heads with respect to the deepest shared convolutional layer (*conv2*), denoted as θ_{shared}:

$$\cos(\theta_{i,j}) = \frac{\nabla_{\theta_{\text{shared}}}\mathcal{L}_i \cdot \nabla_{\theta_{\text{shared}}}\mathcal{L}_j}{\|\nabla_{\theta_{\text{shared}}}\mathcal{L}_i\|\|\nabla_{\theta_{\text{shared}}}\mathcal{L}_j\|}, \tag{16}$$

where $\mathcal{L}_i$ and $\mathcal{L}_j$ denote the loss functions associated with the ith and jth auxiliary heads, respectively; $\nabla_{\theta_{\text{shared}}}(\cdot)$ represents the gradient operator with respect to the shared parameters; and $\|\cdot\|$ signifies the L_2 norm of the gradient vector. A gradient conflict occurs when $\cos(\theta_{i,j}) < 0$ (an angle $> 90°$), indicating that optimizing one head inherently degrades the representation for another.

Fig. 9 illustrates the average gradient conflict ratios across multiple independent runs on both the ShipsEar and DeepShip datasets, with shaded regions denoting the standard deviation.

As shown in Fig. 9(a), the ShipsEar dataset amplifies optimization challenges for standard joint MTL. The gradient conflict ratio initially reaches approximately 60% with high variance and persistently oscillates around 20% in later stages. Similarly, on the DeepShip dataset (Fig. 9(b)), standard MTL still exhibits a continuous conflict ratio of 25% to 30% despite narrower variance. These persistent conflicts limit the representation capacity of the shared backbone and hinder convergence.

In contrast, the proposed STMH strategy maintains a 0% gradient conflict ratio across both datasets under our measurement protocol. STMH adopts a sequential residual supervision paradigm where predecessor heads are frozen as residual anchors and only a single ac-

Table 6
Generalizability analysis across diverse encoders.

Backbone	Accuracy (%)		Precision (%)		Recall (%)		F1-score (%)		Average (%)		
	Base	Ours	Base	Ours	Base	Ours	Base	Ours	Base	Ours	Imp
ShipsEar											
MobileNetV3	65.87 ± 2.58	69.60 ± 2.48	64.34 ± 2.71	68.67 ± 2.45	61.40 ± 3.09	67.08 ± 2.63	62.21 ± 2.83	67.27 ± 2.46	63.46 ± 2.69	68.15 ± 2.46	+4.69
ShuffleNetV2	67.85 ± 2.05	68.31 ± 2.48	67.30 ± 2.33	67.78 ± 3.19	65.16 ± 2.02	67.88 ± 3.33	65.60 ± 2.05	67.43 ± 3.04	66.48 ± 1.98	67.85 ± 2.96	+1.37
ResNet18	66.84 ± 2.80	69.43 ± 2.41	63.44 ± 3.53	69.02 ± 3.47	63.60 ± 2.95	67.15 ± 2.54	63.05 ± 3.14	66.74 ± 2.45	64.23 ± 3.05	68.09 ± 2.52	+3.86
SE-ResNet18	70.13 ± 1.78	71.71 ± 2.88	67.98 ± 2.02	71.29 ± 3.30	67.08 ± 2.23	69.66 ± 3.31	67.01 ± 2.17	69.58 ± 3.27	68.05 ± 2.03	70.56 ± 3.15	+2.51
EfficientNet	68.99 ± 2.59	70.48 ± 2.49	67.99 ± 2.94	69.36 ± 2.76	68.36 ± 2.77	69.12 ± 3.00	67.20 ± 3.02	68.66 ± 3.19	68.14 ± 2.74	69.41 ± 2.78	+1.27
DenseNet121	70.04 ± 1.60	72.40 ± 2.76	70.04 ± 1.24	71.62 ± 2.67	66.99 ± 2.40	68.59 ± 3.71	67.53 ± 2.17	69.09 ± 3.48	68.65 ± 1.65	70.43 ± 3.08	+1.78
Xception	68.72 ± 2.39	**73.10 ± 3.03**	64.73 ± 3.03	**71.89 ± 3.31**	66.39 ± 2.39	**72.02 ± 3.23**	65.23 ± 2.77	**71.53 ± 3.33**	66.27 ± 2.62	**72.13 ± 3.21**	**+5.86**
DeepShip											
MobileNetV3	71.24 ± 2.25	72.12 ± 0.76	71.21 ± 1.92	72.45 ± 0.69	70.61 ± 1.79	71.95 ± 0.76	70.70 ± 1.72	72.08 ± 0.73	70.94 ± 1.88	72.15 ± 0.73	+1.21
ShuffleNetV2	72.43 ± 0.35	73.35 ± 0.80	72.89 ± 0.50	73.57 ± 0.73	72.45 ± 0.34	73.19 ± 0.76	72.40 ± 0.38	73.28 ± 0.76	72.54 ± 0.37	73.35 ± 0.76	+0.81
ResNet18	72.12 ± 1.48	74.78 ± 0.74	73.03 ± 1.26	75.20 ± 0.65	72.00 ± 1.48	74.72 ± 0.69	72.20 ± 1.44	74.80 ± 0.72	72.34 ± 1.40	74.88 ± 0.68	+2.54
SE-ResNet18	72.99 ± 1.53	75.53 ± 0.76	73.64 ± 1.38	75.85 ± 0.80	73.01 ± 1.47	75.29 ± 0.72	73.01 ± 1.44	75.41 ± 0.66	73.16 ± 1.44	75.52 ± 0.73	+2.36
EfficientNet	72.79 ± 0.74	73.88 ± 0.82	73.08 ± 1.04	74.25 ± 0.76	72.71 ± 0.81	73.70 ± 0.83	72.71 ± 0.79	73.84 ± 0.80	72.82 ± 0.84	73.92 ± 0.79	+1.10
DenseNet121	73.00 ± 1.80	74.10 ± 0.81	73.48 ± 1.68	74.25 ± 1.04	72.89 ± 1.76	73.79 ± 0.86	73.03 ± 1.75	73.69 ± 1.08	73.10 ± 1.74	73.96 ± 0.83	+0.86
Xception	72.89 ± 1.79	**76.14 ± 0.90**	73.72 ± 1.86	**76.26 ± 0.72**	72.92 ± 1.84	**76.03 ± 0.92**	73.01 ± 1.82	**76.06 ± 0.88**	73.13 ± 1.82	**76.12 ± 0.84**	**+2.99**

'Imp' denotes the absolute percentage-point improvement in the Average score.

tive head propagates gradients to target unresolved classification errors. This design avoids inter-head gradient conflicts that typically arise in joint multi-head optimization. Consequently, the analysis suggests that STMH reduces optimization interference relative to standard joint MTL, enabling the Xception backbone to learn more robust and category-consistent acoustic features.

5.6. Generalizability analysis across diverse encoders

To verify that the proposed STMH strategy is not exclusively tied to the Xception architecture, we instantiate it on six representative CNN backbones, covering lightweight networks and standard architectures. As summarized in Table 6, the STMH protocol yields consistent improvements across all tested backbones on both datasets, indicating its backbone-agnostic behavior within CNN-based encoders.

On the ShipsEar dataset, STMH serves as an effective regularizer. Lightweight models benefit notably, for example, MobileNetV3 observes a +4.69 pp increase in its average score. Standard high-capacity models like ResNet18 and SE-ResNet18 also achieve substantial gains of +3.86 pp and +2.51 pp, respectively. The Xception backbone records the highest overall improvement (+5.86 pp), supporting its selection as our default architecture. These margins indicate that intermediate sequential supervision aids in discovering discriminative features when data diversity is limited.

On the DeepShip dataset, baseline models have already captured a majority of the acoustic feature variance from the extensive data (e.g., baseline ResNet18 reaches 72.34% average score). Consequently, the margin for residual error correction is narrower. However, STMH still provides reliable enhancements. While lightweight models like MobileNetV3 achieve moderate gains (+1.21 pp), architectures with advanced feature representation capabilities—such as SE-ResNet18 and Xception —experience further improvements of +2.36 pp and +2.99 pp, respectively. Furthermore, ResNet18 secures a +2.54 pp improvement, confirming that the STMH framework effectively elevates the performance ceiling of standard CNNs in complex marine acoustic classification.

5.7. Statistical significance analysis

To complement the averaged performance metrics and quantify the uncertainty of the observed improvements, we conduct a paired statistical analysis using 10 matched runs on ShipsEar and 5 matched runs on DeepShip. Accuracy is selected as the primary inferential metric because the principal performance claims of this study are based on recognition accuracy. Across independent runs, different random seeds were used to account for variability arising from initialization, mini-batch sampling, and stochastic optimization. Within each matched pair, however, the baseline and its STMH-enhanced counterpart shared exactly the same recording-level partition and random seed, thereby ensuring a controlled paired comparison.

For the ith matched run, the paired accuracy difference is defined as

$$d_i = \mathrm{Acc}_{\mathrm{STMH},i} - \mathrm{Acc}_{\mathrm{baseline},i}. \tag{17}$$

The mean paired improvement and its nominal 95% confidence interval (CI) are calculated as

$$\bar{d} \pm t_{0.975,n-1} \frac{s_d}{\sqrt{n}}, \tag{18}$$

where $\bar{d}$ and s_d denote the mean and standard deviation of the paired differences, respectively. A two-sided paired t-test is then conducted using

$$t = \frac{\bar{d}}{s_d/\sqrt{n}}. \tag{19}$$

The raw p-value evaluates an individual backbone comparison. Because STMH is compared with its baseline across seven CNN backbones, the Holm step-down correction is applied separately within each dataset to control the family-wise error rate at $\alpha = 0.05$. Therefore, the final statistical significance is determined using the Holm-adjusted p-value. We additionally report Cohen's $d_z = \bar{d}/s_d$ to quantify the standardized magnitude of each paired improvement. Table 7 reports the mean paired gains, nominal 95% CIs, raw and Holm-adjusted p-values, and Cohen's d_z. STMH produces positive mean gains across all evaluated backbones, although not every numerical improvement remains significant after correction for multiple comparisons.

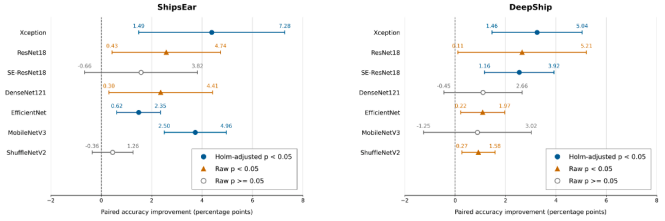
On ShipsEar, STMH-Xception improves Xception by +4.38 percentage points, with a 95% CI of [1.49, 7.28], a raw p-value of 0.0076, a Holm-adjusted p-value of 0.0380, and an effect size of $d_z = 1.08$. The paired differences are positive in all ten matched runs. Xception, EfficientNet, and MobileNetV3 remain significant after Holm correction. ResNet18 and DenseNet121 are significant only before correction, whereas SE-ResNet18 and ShuffleNetV2 remain statistically inconclusive.

On DeepShip, STMH-Xception improves Xception by +3.25 percentage points, with a 95% CI of [1.46, 5.04], a raw p-value of 0.0073, a Holm-adjusted p-value of 0.0482, and an effect size of $d_z = 2.25$. Only Xception

Table 7
Paired statistical analysis of accuracy improvements introduced by STMH. Holm-adjusted p-values below 0.05 are shown in bold.

Backbone	Gain (pp)	95% CI	Raw p	Holm p	d_z
ShipsEar					
Xception	+4.38	[1.49, 7.28]	0.0076	**0.0380**	1.08
ResNet18	+2.58	[0.43, 4.74]	0.0240	0.0960	0.86
SE-ResNet18	+1.58	[−0.66, 3.82]	0.1453	0.2905	0.50
DenseNet121	+2.36	[0.30, 4.41]	0.0290	0.0960	0.82
EfficientNet	+1.48	[0.62, 2.35]	0.0038	**0.0226**	1.22
MobileNetV3	+3.73	[2.50, 4.96]	7.36×10^{-5}	$\mathbf{5.16 \times 10^{-4}}$	2.17
ShuffleNetV2	+0.45	[−0.36, 1.26]	0.2403	0.2905	0.40
DeepShip					
Xception	+3.25	[1.46, 5.04]	0.0073	**0.0482**	2.25
ResNet18	+2.66	[0.11, 5.21]	0.0444	0.1333	1.29
SE-ResNet18	+2.54	[1.16, 3.92]	0.0069	**0.0482**	2.29
DenseNet121	+1.10	[−0.45, 2.66]	0.1193	0.2386	0.88
EfficientNet	+1.09	[0.22, 1.97]	0.0253	0.1010	1.56
MobileNetV3	+0.88	[−1.25, 3.02]	0.3156	0.3156	0.51
ShuffleNetV2	+0.92	[0.27, 1.58]	0.0176	0.0878	1.74

Note: CIs are nominal 95% intervals for individual paired comparisons. Final significance is determined using the Holm-adjusted p-value.



(a) ShipsEar (b) DeepShip

Fig. 10. Paired accuracy improvements introduced by STMH across CNN backbones on the ShipsEar and DeepShip datasets. Points denote the mean paired improvements, and horizontal bars represent nominal 95% confidence intervals. The dashed vertical line at zero indicates no mean improvement. Blue filled circles denote comparisons with Holm-adjusted $p < 0.05$, orange triangles denote raw $p < 0.05$ but Holm-adjusted $p \geq 0.05$, and gray open circles denote raw $p \geq 0.05$.

and SE-ResNet18 remain significant after Holm correction. ResNet18, EfficientNet, and ShuffleNetV2 are significant only before correction, while DenseNet121 and MobileNetV3 remain statistically inconclusive.

To complement the tabulated results, Fig. 10 visualizes the magnitude, direction, and uncertainty of the paired improvements. Each point denotes the mean gain, and each horizontal line represents its nominal 95% CI. Because the intervals are nominal, excluding zero indicates significance in the raw test but does not necessarily imply significance after Holm correction.

As illustrated in Fig. 10, all point estimates lie to the right of the zero-improvement line. The confidence-interval widths also show that the uncertainty varies across backbones and is generally more pronounced when fewer matched runs are available.

Overall, the paired analysis supports the principal finding that STMH-Xception improves upon Xception on both datasets after controlling for multiple comparisons. STMH also yields positive mean gains across the remaining CNN backbones, although their statistical strength varies with the effect magnitude, run-to-run variability, and number of matched runs.

5.8. Computational efficiency analysis

To address the trade-off between representational capacity and computational efficiency, we quantify the parameter count, FLOPs, and inference latency of the proposed STMH strategy across diverse backbones. As detailed in Table 8, the integration of auxiliary heads introduces minimal computational overhead while yielding substantial performance gains.

Overall, STMH improves recognition accuracy across all evaluated backbones on both datasets. The accuracy gains range from 0.46 to 4.38 percentage points on ShipsEar and from 0.88 to 3.25 percentage points on DeepShip. Meanwhile, the parameter increase ranges from only 0.01% to 0.40%, and the additional FLOPs remain below approximately 0.05%. These results show that the performance improvements are generally achieved with limited structural overhead, although the measured wall-clock latency remains backbone-dependent.

For our core architecture, STMH-Xception, the sequential multi-head design increases the total parameter count and FLOPs by a mere 0.03%. The corresponding inference latency penalty is sub-millisecond (+0.67 ms). In exchange for this minimal cost, the model achieves absolute accuracy improvements of 4.38 and 3.25 percentage points on the ShipsEar and DeepShip datasets, respectively.

A closer comparison of the single-head backbones shows that FLOPs alone do not fully determine recognition performance. Under the same terminal classification setting, vanilla SE-ResNet18 achieves higher accuracy than vanilla Xception on both datasets despite requiring fewer FLOPs. One possible explanation is that the residual shortcuts and squeeze-and-excitation channel recalibration of SE-ResNet18 provide an inductive bias that emphasizes informative frequency-channel responses in Mel-spectrograms. Such a property is useful for UATR because discriminative acoustic features are often distributed across different frequency bands. Xception instead employs depthwise separable convolutions to construct hierarchical representations efficiently, but its higher FLOPs do not necessarily translate into better recognition under single-head supervision. The subsequent performance improvement obtained by STMH-Xception suggests that these hierarchical feature stages can be utilized more effectively through stage-wise supervision and sequential residual correction.

To examine whether the performance difference is primarily caused by computational scale, we construct a FLOP-matched SE-ResNet18-Wide by increasing the channel scale of SE-ResNet18. The resulting model requires 0.5840 G FLOPs, which closely matches the 0.5784 G FLOPs of vanilla Xception. Widening SE-ResNet18 improves its accuracy from 70.13% to 70.30% on ShipsEar and from 72.99% to 74.08% on DeepShip. The larger improvement on DeepShip suggests that additional model capacity is more beneficial when sufficient training data are available, whereas its effect is limited under the data-scarce ShipsEar setting.

More importantly, applying STMH to SE-ResNet18-Wide further improves the accuracy from 70.30% to 72.06% on ShipsEar and from 74.08% to 75.15% on DeepShip, while FLOPs change only from 0.5840 G to 0.5842 G. Similar improvements are observed for the original SE-ResNet18. Furthermore, under a comparable FLOP budget, STMH-Xception reaches 73.10% and 76.14% accuracy, exceeding SE-ResNet18-Wide + STMH by 1.04 and 0.99 percentage points on ShipsEar and DeepShip, respectively. These results indicate that STMH provides optimization-level benefits beyond simply increasing backbone capacity. Its later heads progressively correct the prediction errors remaining from earlier stages, allowing performance to improve without materially increasing computational complexity.

The small structural overhead is also observed across the other backbones. For lightweight networks such as MobileNetV3 and ShuffleNetV2, the parameter increase remains below 0.5%, and the additional FLOPs are negligible, thereby preserving their intended structural efficiency. For SE-ResNet18 and DenseNet121, the slightly negative latency differences (−0.05 ms and −1.47 ms) should not be interpreted as computational reductions, but rather as wall-clock fluctuations associated with CPU scheduling and measurement variability. For EfficientNet, the parameter increase is only 0.01% and the FLOPs remain unchanged at the reported precision, whereas the measured latency increases by 12.07 ms. This discrepancy indicates that FLOPs do not fully capture operator scheduling, memory access, and implementation-level overhead. Therefore, STMH introduces consistently small parameter and FLOP over-

Table 8
Comparison of computational overhead and performance across diverse backbones.

Backbone	Method	Params (M) ↓	FLOPs (G) ↓	Inference Time (ms/sample) ↓	Accuracy (%) ↑	
					ShipsEar	DeepShip
Xception	Standard	20.815	0.5784	16.85	68.72 ± 2.39	72.89 ± 1.79
	+ STMH (Ours)	20.822	0.5786	17.52	**73.10 ± 3.03**	**76.14 ± 0.90**
	Increment (Δ)	*+0.03%*	*+0.03%*	*+0.67*	*+4.38*	*+3.25*
ResNet18	Standard	11.172	0.4271	7.71	66.84 ± 2.80	72.12 ± 1.48
	+ STMH (Ours)	11.176	0.4272	8.61	**69.43 ± 2.41**	**74.78 ± 0.74**
	Increment (Δ)	*+0.03%*	*+0.02%*	*+0.90*	*+2.59*	*+2.66*
SE-ResNet18	Standard	11.259	0.4274	11.30	70.13 ± 1.78	72.99 ± 1.53
	+ STMH (Ours)	11.263	0.4275	11.25	**71.71 ± 2.88**	**75.53 ± 0.76**
	Increment (Δ)	*+0.03%*	*+0.02%*	*−0.05**	*+1.58*	*+2.54*
SE-ResNet18-Wide	Standard	15.424	0.5840	17.02	70.30 ± 1.79	74.08 ± 0.14
	+ STMH (Ours)	15.428	0.5842	17.85	**72.06 ± 1.77**	**75.15 ± 0.30**
	Increment (Δ)	*+0.03%*	*+0.03%*	*+0.83*	*+1.76*	*+1.07*
DenseNet121	Standard	6.952	0.6898	22.28	70.04 ± 1.60	73.00 ± 1.80
	+ STMH (Ours)	6.963	0.6902	20.81	**72.40 ± 2.76**	**74.10 ± 0.81**
	Increment (Δ)	*+0.15%*	*+0.05%*	*−1.47**	*+2.36*	*+1.10*
EfficientNet	Standard	20.182	4.005	51.13	68.99 ± 2.59	72.79 ± 0.74
	+ STMH (Ours)	20.184	4.005	63.20	**70.48 ± 2.49**	**73.88 ± 0.82**
	Increment (Δ)	*+0.01%*	*+0.00%*	*+12.07*	*+1.49*	*+1.09*
MobileNetV3	Standard	5.462	0.2190	5.56	65.87 ± 2.58	71.24 ± 2.25
	+ STMH (Ours)	5.464	0.2191	6.10	**69.60 ± 2.48**	**72.12 ± 0.76**
	Increment (Δ)	*+0.03%*	*+0.04%*	*+0.54*	*+3.73*	*+0.88*
ShuffleNetV2	Standard	1.257	0.0198	1.70	67.85 ± 2.05	72.43 ± 0.35
	+ STMH (Ours)	1.262	0.0198	1.83	**68.31 ± 2.48**	**73.35 ± 0.80**
	Increment (Δ)	*+0.40%*	*+0.00%*	*+0.13*	*+0.46*	*+0.92*

**Note:* Marginal fluctuations in CPU inference time are within the standard measurement noise and scheduling variance.

head, while its actual wall-clock latency remains backbone-dependent. Crucially, for compound-scaled heavy architectures like EfficientNet, our optimized lightweight auxiliary heads ensure that the theoretical computational complexity remains unchanged at the reported precision (+0.00% FLOPs), alongside a negligible parameter increase (+0.01%). Ultimately, STMH introduces negligible parameter and FLOP overhead, while the actual wall-clock latency remains backbone-dependent.

Overall, the FLOP-matched comparison in Table 8 clarifies the relationship among backbone architecture, computational complexity, and STMH optimization. Although SE-ResNet18 may benefit from its residual connections and channel-recalibration mechanism, and widening the backbone improves performance to some extent, STMH provides additional gains with almost unchanged parameter and FLOP requirements. These findings indicate that the performance of STMH-Xception cannot be explained by computational scale alone, but instead reflects the interaction between Xception's hierarchical separable feature extraction and the proposed step-wise residual correction mechanism. From a deployment perspective, STMH-Xception retains a single shared backbone and employs only lightweight auxiliary heads, avoiding multiple independent models or ensemble inference. This design improves recognition performance while preserving computational efficiency, supporting its deployment potential in real-time passive sonar and AUV-based monitoring systems.

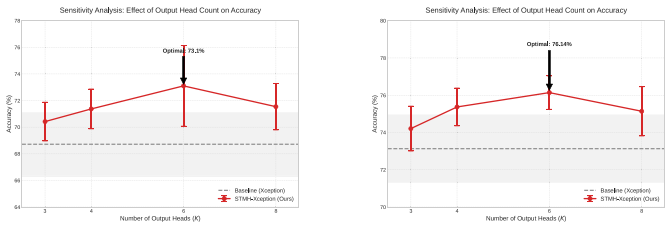
5.9. Hyperparameter sensitivity analysis

A critical architectural hyperparameter in the proposed STMH framework is the total number of auxiliary output heads (K). To evaluate its impact and verify that the optimal configuration is dataset-agnostic, we conduct a sensitivity analysis by varying $K \in \{3, 4, 6, 8\}$ on both the ShipsEar and DeepShip datasets. The results are compared against the single-head Xception baseline.

As illustrated in Fig. 11, introducing sequential auxiliary heads ($K \geq 3$) consistently yields notable accuracy improvements over the baseline across both data scales. This indicates that intermediate stage-wise supervision is effective for deep feature extraction.

On the ShipsEar dataset (Fig. 11(a)), the classification accuracy progressively increases with K and peaks at $K = 6$, achieving an optimal accuracy of 73.10% and an F1-score of 71.53%. A consistent trend is observed on the DeepShip dataset (Fig. 11(b)), where the performance similarly maximizes at $K = 6$, reaching 76.14%.

When the head count is further increased to $K = 8$, the accuracy experiences a slight degradation on both datasets (e.g., dropping to 71.54% on ShipsEar). This phenomenon suggests that while intermediate supervision effectively regularizes shallow layers, an overly dense insertion of auxiliary heads imposes rigid constraints. Such dense constraints can disrupt the natural, hierarchical progression of feature abstractions within the backbone network. Based on this consistent trade-off between representation granularity and optimization stability across different data scales, we adopt $K = 6$ as the default configuration for all experiments in this study.



(a) ShipsEar Dataset (b) DeepShip Dataset

Fig. 11. Sensitivity analysis of the output head count (K) on classification accuracy. The performance peaks consistently at $K = 6$ for both the (a) ShipsEar and (b) DeepShip datasets, indicating that the optimal STMH configuration is robust. Error bars indicate the standard deviation.

6. Conclusion

This paper proposes STMH-Xception to address data scarcity and performance saturation in UATR. STMH reformulates deep supervision as a sequential residual refinement process, in which auxiliary heads are optimized stage by stage to correct the residual prediction errors left by their predecessors. Through step-wise freezing and cumulative-logit correction, STMH reduces the inter-head gradient interference associated with joint multi-head optimization and implements boosting-inspired refinement within a single shared CNN. Evaluations across two data scales demonstrate the effectiveness of the proposed framework. STMH-Xception achieves accuracies of 73.10% on ShipsEar and 76.14% on DeepShip, obtaining the highest accuracy among the evaluated CNN, Mixture-of-Experts, and Transformer baselines. Visualization analyses indicate improved inter-class separability, while experiments across representative CNN encoders show consistent mean improvements, supporting STMH as a CNN-backbone-agnostic strategy within the evaluated scope. Moreover, STMH introduces negligible parameter and FLOP overhead, supporting its deployment potential in resource-constrained marine monitoring systems such as AUVs. Therefore, the observed gains extend beyond mean accuracy, indicating the potential to reduce misclassification risk and improve recognition reliability in long-term underwater acoustic monitoring.

Several limitations remain. Although the colored-noise SNR experiments provide evidence of robustness to unseen noise perturbations, additive noise at different SNR levels cannot fully reproduce complex underwater propagation effects, particularly strong reverberation. Moreover, the current generalizability evidence is limited to CNN-based encoders, and the applicability of STMH to non-CNN architectures has not yet been empirically validated. Future work will evaluate STMH under more realistic reverberant conditions and investigate its extension to Transformer and RNN based architectures.

CRediT authorship contribution statement

Chenhong Yan: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Resources, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization; **Shefeng Yan:** Project administration, Investigation, Funding acquisition; **Ruobin Gao:** Supervision, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization; **Yang Yu:** Validation, Supervision, Software, Resources, Project administration, Investigation; **Guang Pan:** Supervision, Resources, Investigation, Funding acquisition; **Jinpeng Xu:** Validation, Resources; **Ponnuthurai N. Suganthan:** Methodology, Investigation, Conceptualization.

Declaration of competing interests

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is supported by the General Program of the National Natural Science Foundation of China: 62571448; National Natural Science Fund for Excellent Young Scientists Fund Program(Overseas): 5113250157; National Natural Science Foundation of China: 62192711.

References

Akbarian, H., & Sedaaghi, M. H. (2025). Underwater acoustic target recognition in passive sonar using spectrogram and modified mobilenet network classifier. *International Journal of Swarm Intelligence and Evolutionary Computation*, *14*(2), 428. https://doi.org/10.35248/2090-4908.25.14.428

Bae, J., Kang, M., & Choo, Y. (2025). Entropy-based analysis of influential factors for underwater acoustic target recognition in passive sonar data. *Ocean Engineering*, *342*, 122908.

Cai, Z., & Vasconcelos, N. (2018). Cascade R-CNN: Delving into high quality object detection. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 6154–6162).

Cao, X., Togneri, R., Zhang, X., & Yu, Y. (2018). Convolutional neural network with second-order pooling for underwater target classification. *IEEE Sensors Journal*, *19*(8), 3058–3066.

Chen, J., Han, B., Ma, X., & Zhang, J. (2021). Underwater target recognition based on multi-decision lofar spectrum enhancement: A deep-learning approach. *Future Internet*, *13*(10), 265.

Chen, L., Luo, X., Zhou, H., Shen, Q., Chen, L., & Huan, C. (2025). Underwater acoustic multi-target recognition based on channel attention mechanism. *Ocean Engineering*, *315*, 119841.

Chollet, F. (2017). Xception: Deep learning with depthwise separable convolutions. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 1251–1258).

Cortes, C., Gonzalvo, X., Kuznetsov, V., Mohri, M., & Yang, S. (2017). AdaNet: Adaptive structural learning of artificial neural networks. In *International conference on machine learning (ICML)* (pp. 874–883).

Das, A., Kumar, A., & Bahl, R. (2013). Marine vessel classification based on passive sonar data: The cepstrum-based approach. *IET Radar, Sonar & Navigation*, *7*(1), 87–93.

Deng, J., Yang, X., Liu, L., Shi, L., Li, Y., & Yang, Y. (2023). Real-time underwater acoustic homing weapon target recognition based on a stacking technique of ensemble learning. *Journal of Marine Science and Engineering*, *11*(12), 2305.

Dietterich, T. G. (2000). Ensemble methods in machine learning. In *International workshop on multiple classifier systems* (pp. 1–15). Springer.

Doan, V.-S., Huynh-The, T., & Kim, D.-S. (2020). Underwater acoustic target classification based on dense convolutional neural network. *IEEE Geoscience and Remote Sensing Letters*, *19*, 1–5.

Dong, Y., Shen, X., Jiang, Z., & Wang, H. (2021). Recognition of imbalanced underwater acoustic datasets with exponentially weighted cross-entropy loss. *Applied Acoustics*, *174*, 107740.

Fang, H., Xie, Y., Ren, J., Wang, W., & Xu, J. (2025). A momentum-based adversarial training approach for generalization in underwater acoustic target recognition: An individual-vessel perspective. *The Journal of the Acoustical Society of America*, *157*(5), 3508–3523.

Feng, S., & Zhu, X. (2022). A transformer-based deep learning network for underwater acoustic target recognition. *IEEE Geoscience and Remote Sensing Letters*, *19*, 1–5.

Freund, Y., & Schapire, R. E. (1997). A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, *55*(1), 119–139.

Gao, R., Han, R., Dong, H., Suganthan, P. N., & Yu, Y. (2026). Weighted multi-output randomized neural networks with a feature memory pool for electroencephalogram signals classification. *Applied Soft Computing*, *193*, 114796.

Gao, R., Hu, M., Li, R., Luo, X., Suganthan, P. N., & Tanveer, M. (2025a). Stacked ensemble deep random vector functional link network with residual learning for medium-scale time-series forecasting. *IEEE Transactions on Neural Networks and Learning Systems*, *36*(6), 10833–10843.

Gao, R., Liang, M., Dong, H., Luo, X., & Suganthan, P. N. (2025b). Underwater acoustic signal denoising algorithms: A survey of the state-of-the-art. *IEEE Transactions on Instrumentation and Measurement*, *74*, 1–18.

Gao, R., Yang, S., Yuan, M., Wang, Z., Liang, M., Suganthan, P. N., & Yuen, K. F. (2025c). A dynamic ensemble deep randomized neural network using deep autoregressive features for wave height forecasting with missing values. *IEEE Journal of Oceanic Engineering*, *50*(4), 2895–2909.

Garipov, T., Izmailov, P., Podoprikhin, D., Vetrov, D. P., & Wilson, A. G. (2018). Loss surfaces, mode connectivity, and fast ensembling of deep neural networks. In *Advances in neural information processing systems (neurIPS)*. (*vol. 31*).

Hamdy, M., Al-Ali, A., Suganthan, P. N., & Aly, H. (2026). Hybrid training of deep neural networks with multiple output layers for tabular data classification. *Pattern Recognition*, *170*, 112035.

Hazelwood, R. A., & Connelly, J. (2005). Estimation of underwater noise–a simplified method. *Underwater Technology*, *26*(3), 97–103.

Hong, F., Liu, C., Guo, L., Chen, F., & Haihong, F. (2021). Underwater acoustic target recognition with resnet18 on shipsear dataset. In *2021 IEEE 4th international conference on electronics technology (ICET)* (pp. 1240–1244). IEEE.

Huang, G., Li, Y., Pleiss, G., Liu, Z., Hopcroft, J. E., & Weinberger, K. Q. (2017). Snapshot ensembles: Train 1, get M for free. In *International conference on learning representations (ICLR)*.

Hummel, H. I., van der Mei, R., & Bhulai, S. (2024). A survey on machine learning in ship radiated noise. *Ocean Engineering*, *298*, 117252.

Irfan, M., Jiangbin, Z., Ali, S., Iqbal, M., Masood, Z., & Hamid, U. (2021). DeepShip: An underwater acoustic benchmark dataset and a separable convolution based autoencoder for classification. *Expert Systems with Applications*, *183*, 115270.

Jin, S.-Y., Su, Y., Guo, C.-J., Fan, Y.-X., & Tao, Z.-Y. (2023). Offshore ship recognition based on center frequency projection of improved EMD and KNN algorithm. *Mechanical Systems and Signal Processing*, *189*, 110076.

Jin, A., Yang, S., Lei, M., Zeng, X., & Wang, H. (2025). An effective convolutional and transformer cooperation network for underwater acoustic target recognition. *Engineering Applications of Artificial Intelligence*, *159*, 111791.

Jin, G., Liu, F., Wu, H., & Song, Q. (2020). Deep learning-based framework for expansion, recognition and classification of underwater acoustic signal. *Journal of Experimental & Theoretical Artificial Intelligence*, *32*(2), 205–218.

Lakshminarayanan, B., Pritzel, A., & Blundell, C. (2017). Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in neural information processing systems (neurIPS)*. (*vol. 30*).

Lee, C.-Y., Xie, S., Gallagher, P., Zhang, Z., & Tu, Z. (2015). Deeply-supervised nets. In *Artificial intelligence and statistics* (pp. 562–570). Pmlr.

Li, J., & Yang, H. (2021). The underwater acoustic target timbre perception and recognition based on the auditory inspired deep convolutional neural network. *Applied Acoustics*, *182*, 108210.

Liu, F., Shen, T., Luo, Z., Zhao, D., & Guo, S. (2021). Underwater target recognition using convolutional recurrent neural networks with 3-D mel-spectrogram and data augmentation. *Applied Acoustics*, *178*, 107989.

Lv, J., Lan, H., Qin, A., Sun, T., Shao, D., Gao, F., Yao, J., Avanaki, K., & Nie, L. (2024). Dynamic synthetic-scanning photoacoustic tracking monitors hepatic and renal clearance pathway of exogeneous probes in vivo. *Light: Science & Applications*, *13*(1), 304. https://doi.org/10.1038/s41377-024-01644-6

Malik, A. K., Gao, R., Ganaie, M. A., Tanveer, M., & Suganthan, P. N. (2023). Random vector functional link network: Recent developments, applications, and future directions. *Applied Soft Computing*, *143*, 110377.

Moghimi, M., Belongie, S., Saberian, M. J., Yang, J., Vasconcelos, N., & Li, L.-J. (2016). Boosted convolutional neural networks. In *British machine vision conference (BMVC)*.

Ren, J., Xie, Y., Zhang, X., & Xu, J. (2022). UALF: A learnable front-end for intelligent underwater acoustic classification system. *Ocean Engineering*, *264*, 112394.

Ruder, S. (2017). An overview of multi-task learning in deep neural networks. arxiv:1706.05098, .

Santos-Domínguez, D., Torres-Guijarro, S., Cardenal-López, A., & Pena-Gimenez, A. (2016). Shipsear: An underwater vessel noise database. *Applied Acoustics*, *113*, 64–69.

Najamuddin, Sheikh, U. U., & Sha'ameri, A. Z. (2026). Ensemble deep learning approach for marine vessel classification: Integrating CNN and vision transformers with machinery feature enhancement. *Information Fusion*, *126*, 103570.

Shi, Q., Hu, M., Suganthan, P. N., & Katuwal, R. (2022). Weighting and pruning based ensemble deep random vector functional link network for tabular data classification. *Pattern Recognition*, *132*, 108879.

Tan, M., & Le, Q. (2019). EfficientNet: Rethinking model scaling for convolutional neural networks. In *International conference on machine learning* (pp. 6105–6114). PMLR.

Teerapittayanon, S., McDanel, B., & Kung, H.-T. (2016). BranchyNet: Fast inference via early exiting from deep neural networks. In *2016 23rd international conference on pattern recognition (ICPR)* (pp. 2464–2469). IEEE.

Wang, S., & Zeng, X. (2014). Robust underwater noise targets classification using auditory inspired time–frequency analysis. *Applied Acoustics*, *78*, 68–76.

Wang, X., Meng, J., Liu, Y., Zhan, G., & Tian, Z. (2022). Self-supervised acoustic representation learning via acoustic-embedding memory unit modified space autoencoder for underwater target recognition. *The Journal of the Acoustical Society of America*, *152*(5), 2905–2915.

Wang, X., Wu, P., Li, B., Zhan, G., Liu, J., & Liu, Z. (2024). A self-supervised dual-channel self-attention acoustic encoder for underwater acoustic target recognition. *Ocean Engineering*, *299*, 117305.

Xie, Y., Ren, J., & Xu, J. (2022). Adaptive ship-radiated noise recognition with learnable fine-grained wavelet transform. *Ocean Engineering*, *265*, 112626.

Xie, Y., Ren, J., & Xu, J. (2023). Guiding the underwater acoustic target recognition with interpretable contrastive learning. In *Oceans 2023-limerick* (pp. 1–6). IEEE.

Xie, Y., Ren, J., & Xu, J. (2024). Unraveling complex data diversity in underwater acoustic target recognition through convolution-based mixture of experts. *Expert Systems with Applications*, *249*, 123431.

Xu, J., Xie, Y., & Wang, W. (2023a). Underwater acoustic target recognition based on smoothness-inducing regularization and spectrogram-based data augmentation. *Ocean Engineering*, *281*, 114926.

Xu, K., Xu, Q., You, K., Zhu, B., Feng, M., Feng, D., & Liu, B. (2023b). Self-supervised learning–based underwater acoustical signal classification via mask modeling. *The Journal of the Acoustical Society of America*, *154*(1), 5–15.

Yan, C., Yan, S., Yao, T., Yu, Y., Pan, G., Liu, L., Wang, M., & Bai, J. (2024). A lightweight network based on multi-scale asymmetric convolutional neural networks with attention mechanism for ship-radiated noise classification. *Journal of Marine Science and Engineering*, *12*(1), 130.

Yan, C., Yan, S., Yu, Y., Gao, R., Pan, G., Yang, Y., Yao, T., & Suganthan, P. N. (2026). Boosting-inspired sequential multi-head learning for underwater acoustic target recognition. *Ocean Engineering*, *356*, 125324.

Yan, C., Yu, Y., Yan, S., Yao, T., Yang, C., Liu, L., & Pan, G. (2023). Underwater target recognition using a lightweight asymmetric convolutional neural network. In *Proceedings of the 17th international conference on underwater networks & systems* (pp. 1–8).

Yang, Y., Yao, Q., & Wang, Y. (2024). Underwater acoustic target recognition method based on feature fusion and residual CNN. *IEEE Sensors Journal*, *24*(22), 37342–37357.

Ye, M., Yan, X., Jiang, D., Xiang, L., & Chen, N. (2024). MIFDELN: A multi-sensor information fusion deep ensemble learning network for diagnosing bearing faults in noisy scenarios. *Knowledge-Based Systems*, *284*, 111294.

Zeng, X., Liu, X., Song, G., Wang, D., Luo, H., & Zhang, B. (2020). Adversarial training for underwater target recognition in complex marine conditions. In *2020 IEEE 22nd international conference on high performance computing and communications; IEEE 18th international conference on smart city; IEEE 6th international conference on data science and systems (HPCC/smartcity/DSS)* (pp. 1174–1179). IEEE.

Zhang, L., Wu, D., Han, X., & Zhu, Z. (2016). Feature extraction of underwater target signal using mel frequency cepstrum coefficients based on acoustic vector sensor. *Journal of Sensors*, *2016*(1), 7864213.

Zhang, X., Zhou, X., Lin, M., & Sun, J. (2018a). ShuffleNet: An extremely efficient convolutional neural network for mobile devices. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 6848–6856).

Zhang, Y., Hao, J., & Zeng, Q. (2025). Reducing ambient noise diffusion model for underwater acoustic target. *Neural Networks*, *188*, 107470.

Zhang, Y., Xiang, T., & Lu, H. (2018b). Deep mutual learning. In *Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR)* (pp. 4320–4328).

Zhou, A., Li, X., Zhang, W., Li, D., Deng, K., Ren, K., & Song, J. (2023a). A novel cross-attention fusion-based joint training framework for robust underwater acoustic signal recognition. *IEEE Transactions on Geoscience and Remote Sensing*, *61*, 1–16.

Zhou, A., Li, X., Zhang, W., Zhao, C., Ren, K., Ma, Y., & Song, J. (2023b). An attention-based multi-scale convolution network for intelligent underwater acoustic signal recognition. *Ocean Engineering*, *287*, 115784.